GitLab CI/CD

David THIBAU – 2026

david.thibau@gmail.com

Agenda

Introduction

- DevOps et CI/CD
- Container et DevOps
- La plate-forme Gitlab

Workflow de collaboration

- Groupes, Projets et membres
- Planification et suivi
- Branches et Merge Request

Bases pipelines Gitlab

- Introduction
- Jobs et Runners
- UI pipeline

Syntaxe .gitlab-ci.yml

- Structure du fichier
- Principales directives
- Réutilisation
- Intégration docker
- Environnements et déploiements

Pipeline standard dans Gitlab

- Construction et tests développeur
- Analyses statiques
- Dépôts d’artefacts
- Déploiement et release
- Gestion de l’infrastructure
- AutoDevOps

Introduction

DevOps et CI/CD

La plateforme Gitlab

Pourquoi le DevOps ?

Pour faire évoluer le métier le plus rapidement possible :
Boucle d'amélioration courte

DevOps vise l'alignement des équipes par la réunion des "Dev engineers" et des "Ops engineers" .

- Communication forte entre les équipes
- la montée en compétence des différents profils.

=> Arrivée des profils full-stack

Une équipe DevOps est une équipe entièrement **indépendante** regroupant toutes les compétences nécessaires à un projet : *Métier, Dév, Intégrateur, Testeur, Ops*

Pratiques DevOps

Une approche en continu :

- Tests continus à chaque commit
- Déploiement en continu dans chaque environnement
  La répétition fiabilise le processus ;

Un décalage des tests "vers la gauche".
Autrement dit de tester au plus tôt ;

Une pratique des tests dans un environnement similaire à celui de production ;

Une surveillance étroite de l'exploitation

La production de métriques reflétant la qualité du projet

Une unique source de vérité :

Le dépôt de source inclut le code de build, de test, d’infrastructure et … le code source

« As Code »

Les outils DevOps permettent d’automatiser/exécuter toutes les tâches nécessaires à la construction d’un logiciel de qualité (build, test, analyse statique, provisionnement)

On parle alors de Build As Code, Infrastructure As Code, Pipeline As Code, Load Test As Code, ...

Objectif ultime

- Déployer souvent et rapidement
- Automatisation complète
- Zero-downtime des services
- Possibilité d’effectuer des roll-backs
- Fiabilité constante de tous les environnements
- Possibilité de scaler sans effort
- Créer des systèmes résilients, capable de se reprendre en cas de défaillance ou erreurs

Approche en continu

A chaque ajout de valeur dans le dépôt de source (push), l'intégralité des tâches nécessaires à la mise en service d'un logiciel (intégration, tests, déploiement) sont essayées.

- La majeur partie des tâches sont des tests

En fonction de leurs succès, l'application est déployée dans les différents environnements (intégration, staging, production)

Pipelines

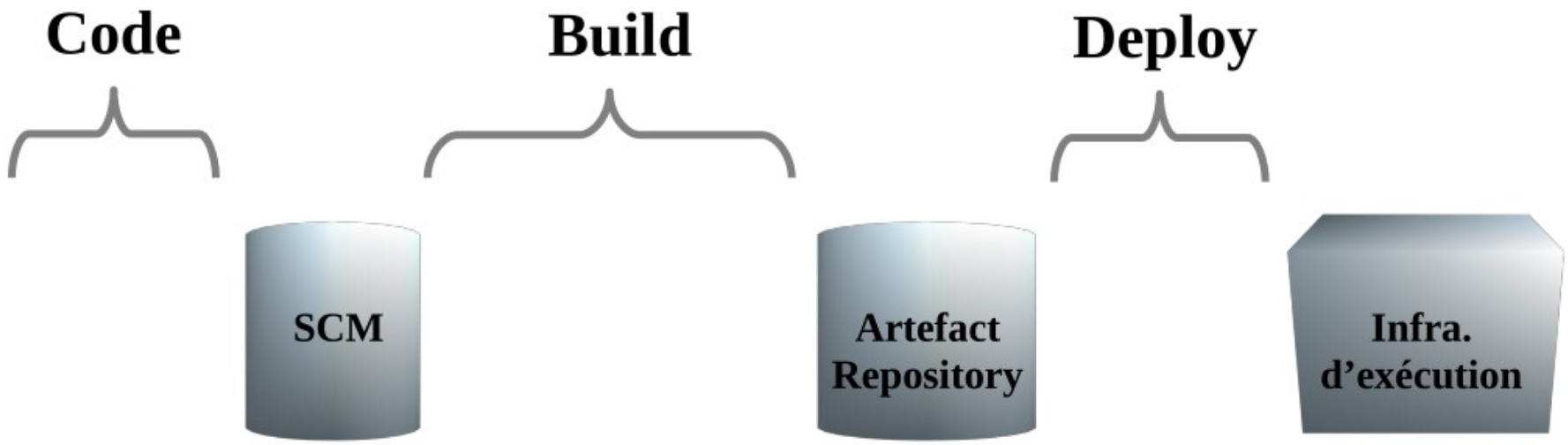
Les étapes de construction automatisées sont séquencées dans une pipeline.

- Une étape est exécutée seulement si les étapes précédentes ont réussi.
- Les plate-formes d’intégration/déploiement continu ont pour rôle de démarrer et observer l’exécution de ces pipelines

Cycle de vie du code Build / Release / Run

La pipeline suit le cycle de vie du code.

1) Le code est testé localement puis poussé dans le dépôt de source
2) Le build construit l'artefact et si le build est concluant stocke une release dans un dépôt d'artefact
3) L'outil de déploiement accède aux différentes release et les déploie sur l'infra d'exécution



Distinction CI/CD

Outil de communication

La Plateforme a pour vocation de publier les résultats des builds :

- Nombre de tests exécutés, succès/échecs
- Couverture des tests
- Complexité, Vulnérabilités du code source
- Performance : Temps de réponse/débit
- Documentation, Release Notes
- ...

Les métriques sont visibles de tous en toute transparence

=> Confiance dans ce qui a été produit

=> Motivation pour s’améliorer

Pipeline et les containers

Plusieurs cas d'usage des Docker dans un contexte de pipeline:

- Utiliser des images pour exécuter les builds
  => Facilite l'exploitation de la plateforme CI/CD
- Construire et pousser des images pendant l'exécution d'une pipeline
  => Permet les déploiements immuables
- Utiliser des images pour exécuter des services nécessaires à une étape de build
  => Test d'intégration nécessitant les services de support (BD, Broker, ...)

Introduction

DevOps et CI/CD

La plateforme Gitlab

Introduction

Gitlab se définit comme une plateforme DevOps complète qui inclut :

- La gestion des codes sources
- Le pilotage de projet agile
- Exécution de pipelines de CI/CD
- La gestion des dépôts artefacts
- La gestion des environnements et infrastructure de déploiement
- La mise à disposition des bonnes pratiques DevOps

Intégration GitLab

Gitlab peut également s'intégrer avec d'autres outils qui pourront prendre une part du Devops

- Annuaire utilisateurs et sécurité (LDAP, OpenID, …)
- Pilotage de projet (Jira, …)
- Communication projet (Slack, Gmail, …)
- Dépôt d'artefact (Nexus, Artifactory, Docker registry,…)
- Outils d'analyse (Sonarqube, …)
- Exécution de pipeline (Jenkins, ...)
- Infrastructure (Terraform, Kubernetes, Cloud)
- …

Installations

Gitlab s'installe sous Linux. Différentes façons :

- Omnibus Gitlab : Packages pour différentes distributions de Linux
- GitLab Helm chart : Version Cloud, installation sous Kubernetes
- Images Docker
- A partir des sources

Également disponible en ligne : gitlab.com

Community vs Enterprise

Les 2 éditions ont le même cœur, l' enterprise edition ajoute du code propriétaire.

Le code propriétaire peut devenir gratuit au fur et à mesure des évolutions

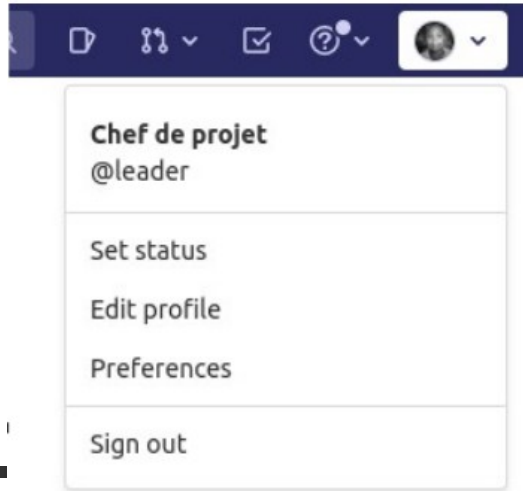
Les versions payantes apportent généralement :

- Des fonctionnalités innovantes
- Des fonctionnalités avancées (Scanners de sécurité par exemple)
- Des fonctionnalités transverses aux projets
- Des facilités d'intégration avec des outils
- Une installation en HA
- Du support 24h/24

Interface utilisateur

2 interfaces utilisateurs sont disponibles :

- Administrateur : Permet de configurer la plateforme, de gérer les utilisateurs, de configurer les runners disponibles et de configurer de façon transverse certains aspects des projets
- Utilisateur :
  - Permet de gérer son compte (infos, crédentiels, notifications, préférences)
  - Permet d’accéder à ses projets



Menu User Settings

Profile : Édition du profil utilisateur

Account : Gestion de l'authentification (Possibilité d'activité le 2 factors)

Applications : Se connecter avec un fournisseur oAuth2

Personal Access Token : Jeton représentant l'utilisateur pouvant être utilisé pour accéder à l'API Gitlab

Emails : Possibilité d'associer plusieurs emails au compte

Password : Modification mot de passe

Notifications : Configurer le niveau de notifications de Gitlab

SSH Keys : Pouvoir accéder au dépôt en ssh et sans mot de passe

GPG Keys : Pouvoir signer des tags

Preferences : Personnalisation de l'UI

Active Sessions : Les sessions actives (Navigateur loggés avec le compte)

Authentication Log : Journal des authentifications

Mise en place clés ssh

La mise en place des clés ssh permet de pouvoir interagir avec le dépôt de source sans avoir à fournir de mot de passe.

2 étapes :

- Créer une paire de clé privé/publique
- Fournir la clé publique à Gitlab via l’interface web

Mise en place

Environnement Linux :

ssh-keygen -t ed25519 -C "email@example.com"

Ou

ssh-keygen -o -t rsa -b 4096 -C "email@example.com"

Copier le contenu de la clé publique (*.pub) dans l'interface Gitlab

Tester avec :

ssh -T git@gitlab.com

Workflows de collaboration

Projets et Membres
Planification et suivi
Branches et MergeRequest

Projets

Un projet Gitlab a vocation à être associé à un dépôt de source Git

Par défaut, tous les utilisateurs Gitlab peuvent créer un projet

3 visibilité sont possibles pour un projet :

- **Public** : Le projet peut être cloné sans authentification.
- **Interne** : Peut être cloné par tout utilisateur authentifié.
- **Privé** : Ne peut être cloné et visible seulement par ses membres

Les projets d’entreprise sont en général privé

Membres

- Les utilisateurs peuvent être affectés à des projets, ils en deviennent membres
- Un membre a un rôle qui lui donne des permissions sur le projet :
  - **Guest** : Créer un ticket
  - **Reporter** : Obtenir le code source
  - **Planner** : Organiser les milestone et les issues
  - **Developer** : Push/Merge/Delete sur les branches non protégées, Merge request sur les autres branches
  - **Maintainer** : Administration de l'équipe, Gestion des branches protégés ou non, Labels,
  - **Owner** : Créateur du projet, a le droit de le supprimer

Groupes

Afin de faciliter la gestion des projets et des membres associés, il est possible de définir des **groupes de projets**.

Tous les projets du groupe hériteront des configurations (Visibilité, membres, ...)

Il sera possible de visualiser toutes les issues et Merge Request des projets du groupe

Les groupes peuvent être hiérarchiques

Attention : Il est dangereux de déplacer un projet existant dans un autre groupe

Menu Groupe Settings

Group Information → Members : Ajout de membres

General : Nom et visibilité

Integration : Intégration à des outils tierces (Slack, JIRA, …)

Access Token : Jeton d'accès à l'API concernant les projets du groupe

Projects : Projets du groupe

Repository : Jetons permettant à des applications tierces de cloner le dépôt, récupérer des artefacts stockés dans Gitlab, nom de la branche par défaut

CI/CD : Définition de variables, de runners, activation/désactivation de AutoDevOps

Applications : Fournisseur oAuth2

Package & Registries : Définition de dépôts d'artefacts, de proxy des dépendances

Création de projet

La création de projet peut se faire à partir de la home page ou de la page d'un groupe

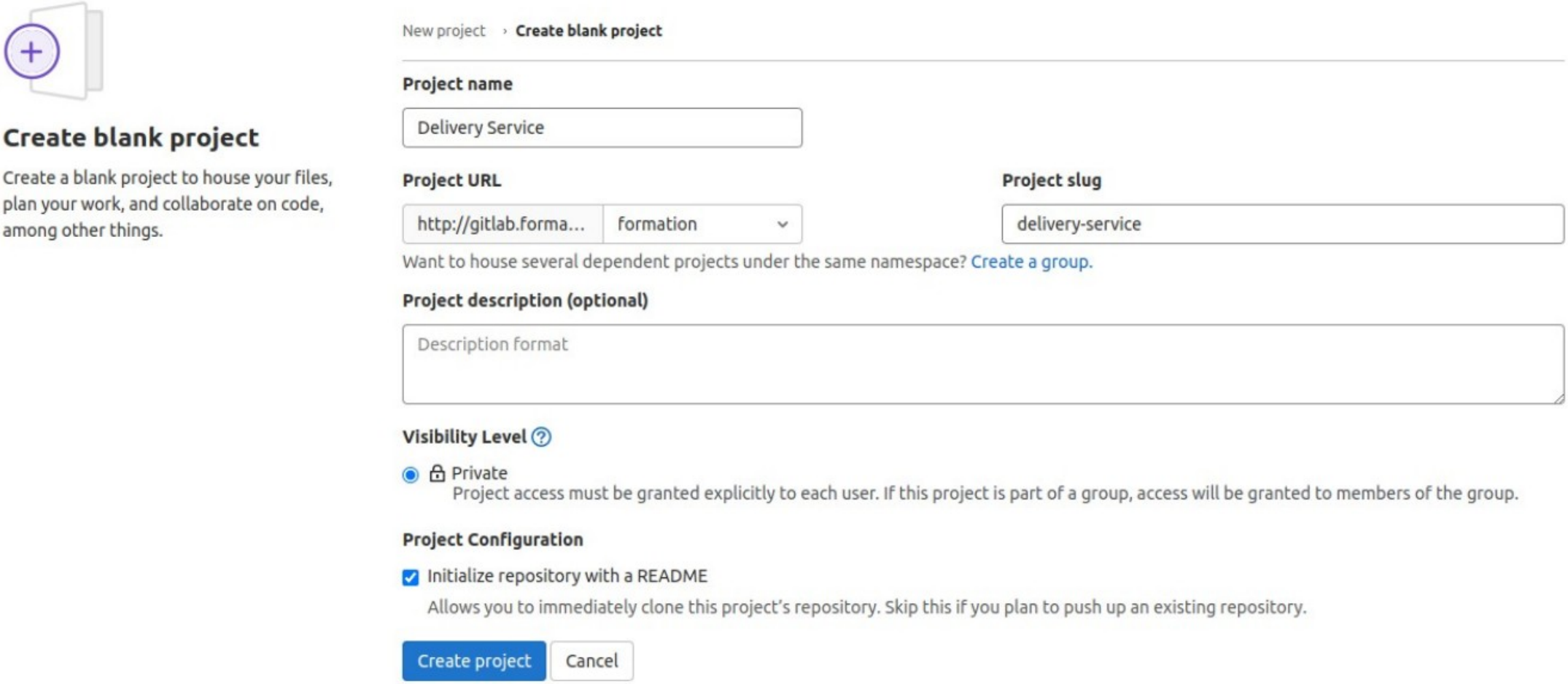
Il peut s'agir :

- D'un projet vierge
- D'un projet à partir d'un gabarit contenant déjà certains fichier
- En important un projet d'un autre dépôt Git

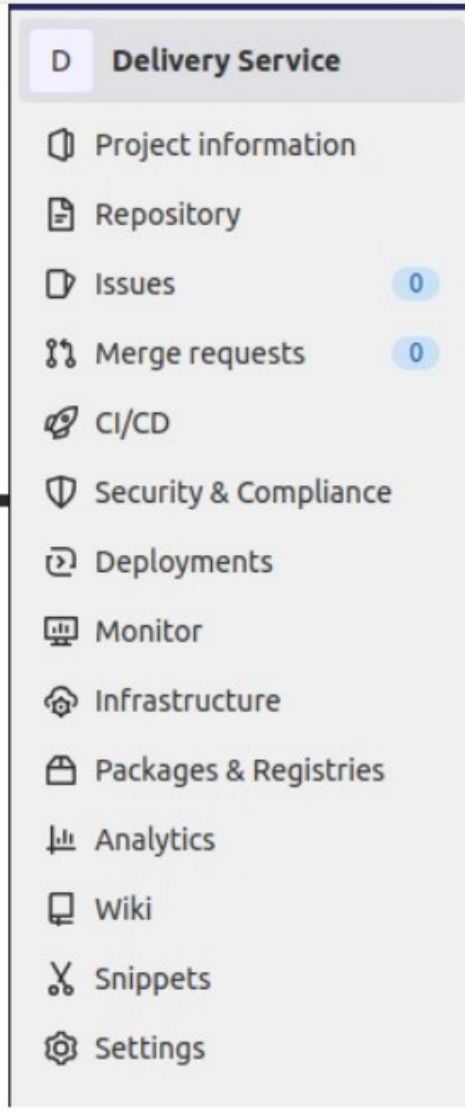
Lors de la création, il faut définir :

- Un nom
- Un project slug qui donnera lieu à une URL d'accès (pas de caractères spéciaux)
- La visibilité
- Si le dépôt Git associé au projet doit être initialisé avec un fichier README

Création projet vierge à partir d’un groupe



Menu Projet



Projects : Activité, Labels et membres

Repository : Navigateur de fichiers, Commits, branches, tags, historique, comparaison, statistiques sur les fichiers du projet

Issues : Gestion des issues, tableau de bord Kanban

Merge requests : Travaux en cours

CI/CD : Historique d'exécution des pipelines

Security & compliance : Rapports sur les détections de vulnérabilités

Deployments : Gestion des environnements de déploiement

Monitor : Information de surveillance du projet

Infrastructure : Cluster Kubernetes associés, Plateforme serverless, Historique de changements Terraform

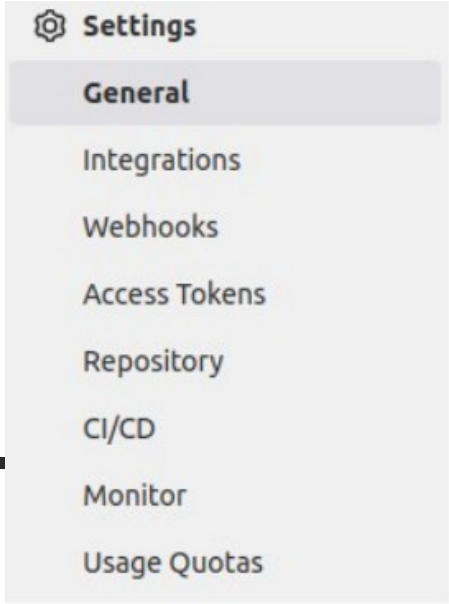
Packages et registries : Accès aux dépôts d'artefacts

Wiki : Documentation annexe

Snippets : Bouts de code

Settings : Configuration

Menu Projet → Settings



General :

- Nom, Classification Topic, Avatar,
- Visibilité projet, Configuration des features (menus accessibles)
- Merge request : Configuration des fusions de branches
- Badges,
- Service Desk : Utilisateurs pouvant envoyer des issues par mail
- Advanced : Suppression, déplacement de projet, …

Integrations : Intégration application tierces

Webhook : Alternative à Integration. Permet d'envoyer un webhook à une application tierce

Access Token : Jeton d'accès pour l'API projet

Repository : Branche par défaut, branches et tags protégées, dépôt miroir, Clé et jetons permettant d'accéder au dépôts et aux packages, Nettoyage du dépôt

CI/CD : Configuration général pipelines, AutoDevOps, Runners, politique de rétention des artefacts, Jeton de déclenchement, …

Monitor : Configuration du monitoring

Usage Quotas : Définition de quotas de stockage

Workflows de collaboration

Projets et Membres

Planification et suivi

Branches et MergeRequest

Issues et Milestones

Gitlab permet de s'adapter à chaque méthodologie agile via les **issues**, les **milestones** et les **epics** dans la version payante

Une issue peut ainsi représenter :

- Une user story
- Un demande d'évolution
- Une déclaration de bug
- Un idée d'amélioration
- Une tâche technique

Elles sont généralement affectés à des **milestones** qui peuvent représenter :

- Une release
- Un sprint
- Une date de livraison
- ....

Les **Epics** permettent de rassembler des issues provenant de différents projets

Visualisation des issues

Les issues peuvent être visualisées via :

- Une **liste** : Toutes les issues du projet avec possibilité de filtrer ou faire des actions par lots (bulk)
- Un **board** : Tableau de bord façon Kanban, permettant de glisser/déposer les issues dans des colonnes représentant le statut de l'issue
- **Epic** : Vision transversale aux projets des issues partageant un thème, un milestone,

Labels

Les **labels** jouent un rôle très important dans Gitlab

Ce sont des petits libellés colorés qui permettent de tagger les objets Gitlab :

Affecter des labels aux issues permet :

- De catégoriser les issues
- De filtrer les listes d’issues
- De créer les tableaux de bord (boards)

Usage des Labels

Gitlab propose des labels par défaut mais il est possible de configurer ses propres labels

Un label peut être défini au niveau

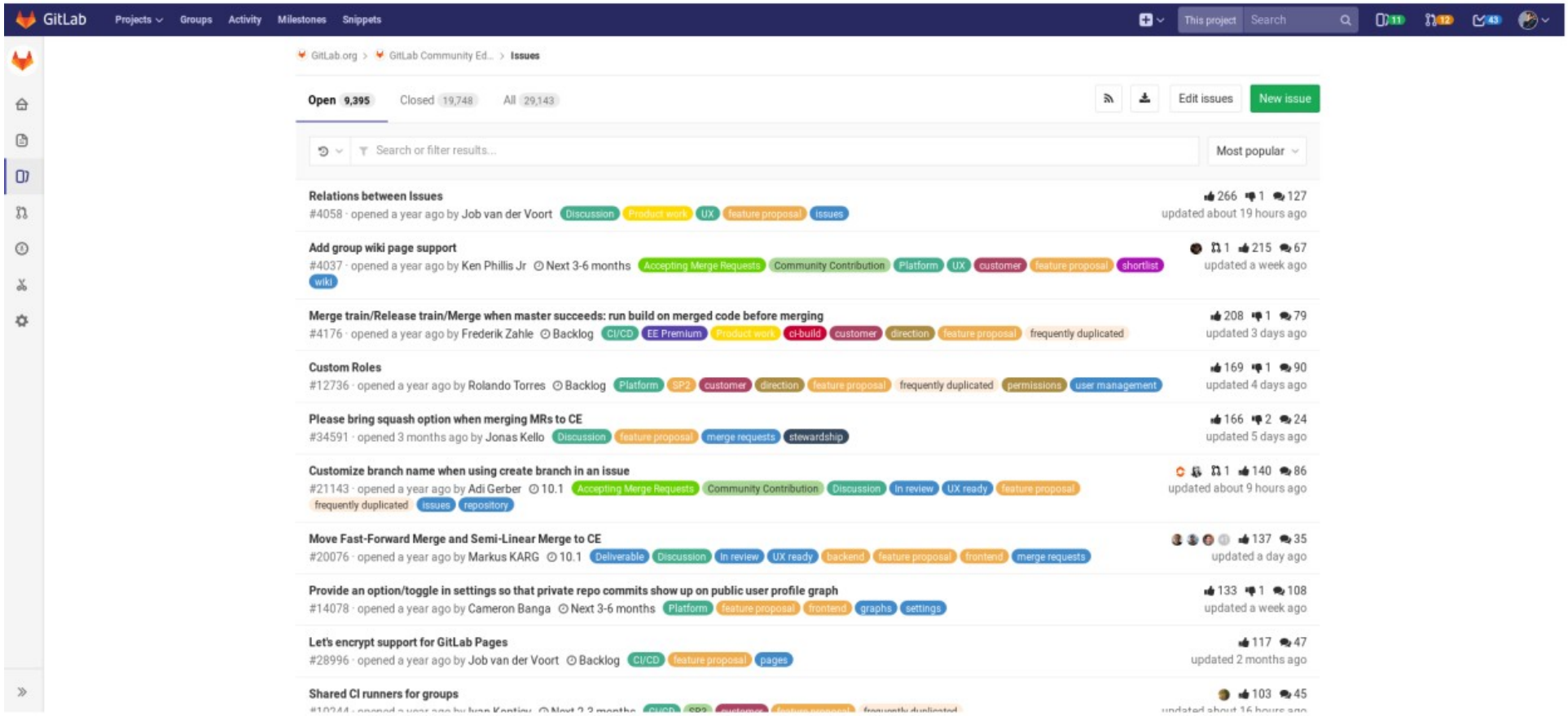
- Groupe : ***Group information > Labels.***
- ou projet : ***Project information > Labels***.

Plusieurs labels peuvent être associés à la même issue

On peut donc créer des labels permettant :

- De typer (Bug, Idée, RFC, User Story, …)
- Indiquer le domaine concerné (Front-end, Back-end, CI/CD,…)
- Indiquer un statut (Review, Duplicate, …)
- …

Exemple : Liste d’issues avec labels



Exemple board

13 To do

- Suppression Formation
  #236 opened by dthibau
  bug
- Déployer sous forme de conteneur Docker
  #226 opened by dthibau
- Implémenter Cache SpringBoot
  #232 opened by dthibau
  enhancement
- Etude - plateforme Kubernetes d'OVH
  #81 opened by dthibau
  enhancement
- Utiliset OpenAPI pour simplifier les développements front-end
  #83 opened by dthibau
  enhancement Tek
- URL permettant de tester toutes les notifications
  #32 opened by dthibau
- Créer un système de tag dans PLBSI V1 et V2
  #61 opened by Vincent-PLB
  Reminder
  2.8
- Donner de la visibilité sur les cours CPF et action co sur la vue

2 In progress

- Inclure des tests automatisés des systèmes dépendants de la base plbsi dans la pipeline Jenkins
  #240 opened by dthibau
  enhancement Tek
- Partenaires : Ajouter un onglet "Contexte" (on le renommera ultérieurement si nécessaire)
  #251 opened by Vincent-PLB
  enhancement
  Partenaire 2.7.3

1 A corriger

- Déplacer une formation trop loin dans une catégorie fait planter le module de réorganisation des formations
  #201 opened by Vincent-PLB
  bug

6 Recette

- Partenaires : Ajouter un encart "Délégation" pour Aurore
  #252 opened by Vincent-PLB
  enhancement
  Partenaire 2.7.3
- Les blocs Remarques du Commerce peuvent-elles afficher la date, l'heure et l'auteur de la dernière modication ?
  #95 opened by Vincent-PLB
  enhancement
  Partenaire 2.7.3
- Partenaires : Augmenter la limite de caractères sur les blocs de texte libre
  #249 opened by Vincent-PLB
  enhancement
  Partenaire 2.7.3
- Système de tooltip
  #255 opened by dthibau
  enhancement
  Partenaire 2.7.3
- Prochain Inter confirmé : ne plus prendre en compte les sessions à 1 participant
  #238 opened by Vincent-PLB
  Kibana
  Partenaire 2.7.3
- Ajouter les blocs remarques dans l'export excel

88 Done

- Certaines formation partenaire référencent des formations qui n'existent plus
  #254 opened by dthibau
  BD invalid Tek
  Partenaire 2.7.3
- Monitoring plbsi
  #225 opened by dthibau
  Tek
  2.7
- Upgrader la version de Java
  #49 opened by dthibau
  Tek
  2.7
- Créer les bases du Suivi des Offres dans PLBSI V2
  #120 opened by Vincent-PLB
  wontfix
- Mettre en place une rotation pour plbsi.log
  #67 opened by dthibau
  Tek
- Créer une maquette pour la page Historique
  #158 opened by Vincent-PLB
  duplicate
  2.5
- En consultation, le texte du champ

Champs d’une issue à la création

Une issue comporte de nombreux champs qui sont pour la plupart optionnels. Les principaux sont :

- Titre : On peut forcer des gabarits
- Types : Issue ou Incident
- Description : Texte riche
- Assignee : Les personnes impliquées
- Due date
- Milestone
- Labels

Une issue peut être crée par tous les membres du projet et même par les utilisateurs si on active la fonctionnalité ***ServiceDesk***

Collaboration autour de l’issue

De nombreuses fonctionnalités de collaboration sont proposées autour de l’issue :

- Threads de discussion et notifications/alertes
- Workflows (Changement de labels/statut)
- Association d’une issue à une Merge Request, et donc aux autres objets associés à la MR:
  - Aux modifications de code, aux commits
  - A la revue de code
  - Aux pipelines, aux résultats des tests automatisés
  - Aux environnements de déploiement pour réceptionner l’issue (Review Apps)

Autres fonctionnalités

Issues liées : Permet d'associer une issue à une autre (Travail préliminaire, contexte, dépendance, doublon)

Crosslinking : Liens vers des objets référençant l'issue. (Commit, Autre Issue ou Merge Request)

- Par exemple un commit
  git commit -m "this is my commit message. Ref #xxx"

Fermeture automatique : Possibilité de fermer les issues automatiquement après un merge request

Gabarits : Créer des issues à partir de gabarits

Édition en mode **bulk**

Import/Export d'issues

API Issues

Atelier 2.2 : Création issues, labels, tableaux de bord, milestone

Workflows de collaboration

Projets et Membres
Planification et suivi
Branches et MergeRequest

Branche par défaut

A la création de projet, GitLab positionne ***main*** comme branche par défaut.

- Peut-être changé dans ***Settings → Repository***.

C'est en général dans la branche par défaut que sont fusionnées les modifications effectuées dans les autres branches

Lors de l'accès aux sources, c'est la branche par défaut qui est affichée

Création de branche

- Plusieurs façons de créer des branches avec Gitlab :
  - **A partir d'une issue**, en créant une Merge Request
    La branche est créé à partir de la branche par défaut
    Elle est dédiée à la résolution de l'issue et est généralement supprimée lorsque l'issue est résolue
  - A partir du **menu** (***Repository → Branches***),
    Il est possible d'indiquer la branche de départ
  - En **commande en ligne**, en poussant une branche locale vers le dépôt

Branche protégée

Les Maintainer peuvent créer/supprimer des branches protégées.

2 permissions sont alors associées :
Project → Settings → Protected branches

- ***Allow to Merge*** : En général le Maintainer
- ***Allow to Push*** : En général le Maintainer

La branche par défaut est une branche protégée.

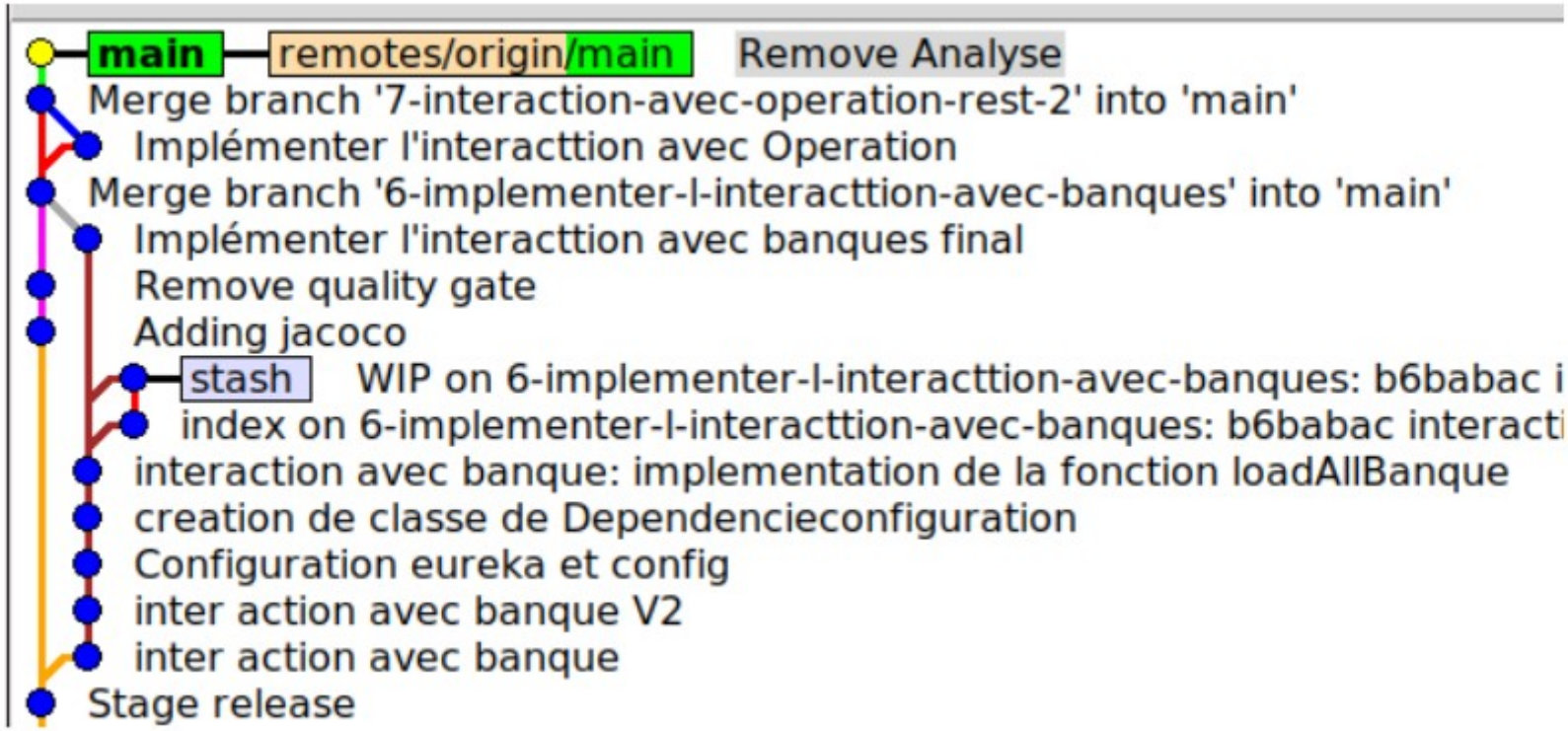
On peut utiliser des wildcards pour protéger plusieurs branches en même temps.
Ex : **-stable, production/**

Workflow de résolution d’issue

Le workflow de résolution d’issue proposé par Gitlab est le suivant :

1. Lorsqu’un développeur prend un issue il active le bouton ***Create Merge Request***. Cela a pour conséquence :
   - La création d’une branche de feature portant le nom de l’issue
   - La création d’un objet MergeRequest qui rassemblera toutes les informations relatives à la résolution de l’issue
2. Le développeur travail en local et effectue régulièrement des push vers Gitlab provoquant l’exécution de pipeline
3. Lorsqu’il pense avoir terminé, il active le lien ***Mark as Ready***
4. Le mainteneur utilise la MR pour :
   - Voir tous les changements
   - Effectuer une revue de code
   - Voir les résultats de la pipeline
5. Il décide alors d’accepter / refuser ou fermer la MR.
6. En cas d’acceptation, la branche de features est fusionnée dans la branche protégée puis généralement supprimée

Exemple


main
remotes/origin/main
Remove Analyse
Merge branch '7-interaction-avec-operation-rest-2' into 'main'
Implémenter l'interacttion avec Operation
Merge branch '6-implementer-l-interacttion-avec-banques' into 'main'
Implémenter l'interacttion avec banques final
Remove quality gate
Adding jacoco
stash
WIP on 6-implementer-l-interacttion-avec-banques: b6babac i
index on 6-implementer-l-interacttion-avec-banques: b6babac interacti
interaction avec banque: implementation de la fonction loadAllBanque
creation de classe de Dependencieconfiguration
Configuration eureka et config
inter action avec banque V2
inter action avec banque
Stage release

Merge Request

Une Merge Request est donc un objet regroupant toutes les informatives relatives à un travail ayant vocation a être fusionné dans une branche protégée.

Le cycle de vie d’un MR est :

- Draft
- Ready
- Merged : Les travaux ont été intégrés
- Closed : Les travaux ont été abandonnés

Resolve "Implémenter l'interacttion avec banques"

Edit | Code

Merged **mouhamadou bamba badjinka** requested to merge `6-implementer-l-interactti…` into `main` 5 days ago

Overview 0 | Commits 6 | Pipelines 7 | Changes 17

Add a to do

Closes #6 (closed)

0 | 0

Pipeline #1231758956 failed

Pipeline failed for `18dc461e` on `6-implementer-l-interacttion-avec-banques` 3 days ago

Approval is optional

Merged by David THIBAU 3 days ago

Revert | Cherry-pick

Merge details

- Changes merged into `main` with `2253fd14`.
- Deleted the source branch.
- Closed #6 (closed)

Pipeline #1231763196 failed

Pipeline failed for `2253fd14` on `main` 3 days ago

Activity

All activity

- **mouhamadou bamba badjinka** requested review from @badjinka 5 days ago
- **mouhamadou bamba badjinka** assigned to @dthibau 5 days ago
- **mouhamadou bamba badjinka** added 1 commit 5 days ago
  - `af44fecd` - inter action avec banque

  Compare with previous version
- **mouhamadou bamba badjinka** added 1 commit 5 days ago
  - `5f8ab2cf` - inter action avec banque V2

  Compare with previous version
- **David THIBAU** added 1 commit 5 days ago
  - `65d26fd4` - Configuration eureka et config

  Compare with previous version

Assignee Edit

David THIBAU

Reviewer Edit

mouhamadou bamba badjinka

Labels Edit

None

Milestone Edit

None

Time tracking

No estimate or time spent

2 Participants

Commentaires et discussions

Des commentaires peuvent être associés aux MR

- Soit au niveau général
- Soit au niveau d'un commit particulier

Un commentaire peut être transformé en **discussion/thread.**

Une discussion/thread regroupe plusieurs échanges et a un statut

- La discussion démarre avec un statut **unresolved**
- Elle se termine avec le statut **resolved**

Il est possible de

- voir toutes les discussions non résolues
- De déplacer les discussions non résolues vers une issue
- D'empêcher la fusion, si une discussion est non résolue
  (***Project → Settings → General → MR***)

Revue de code

Une revue de code consiste à effectuer plusieurs commentaires liés à des lignes de code.

Lors d'une revue de code, le reviewer commence par créer des commentaires visibles uniquement par lui.

Lorsqu'il est prêt, il publie l'ensemble des commentaires en une fois.

- 1) Sélectionner l'onglet ***Changes*** de la MergeRequest
- 2) Sélectionner l'**icône de commentaire** en face du patch
- 3) Écrire le 1er commentaire et activer le bouton ***Start Review***
- 4) Faire d'autres commentaires et activer le bouton ***Add to review***
- 5) A la fin, activer le bouton ***Submit the review***

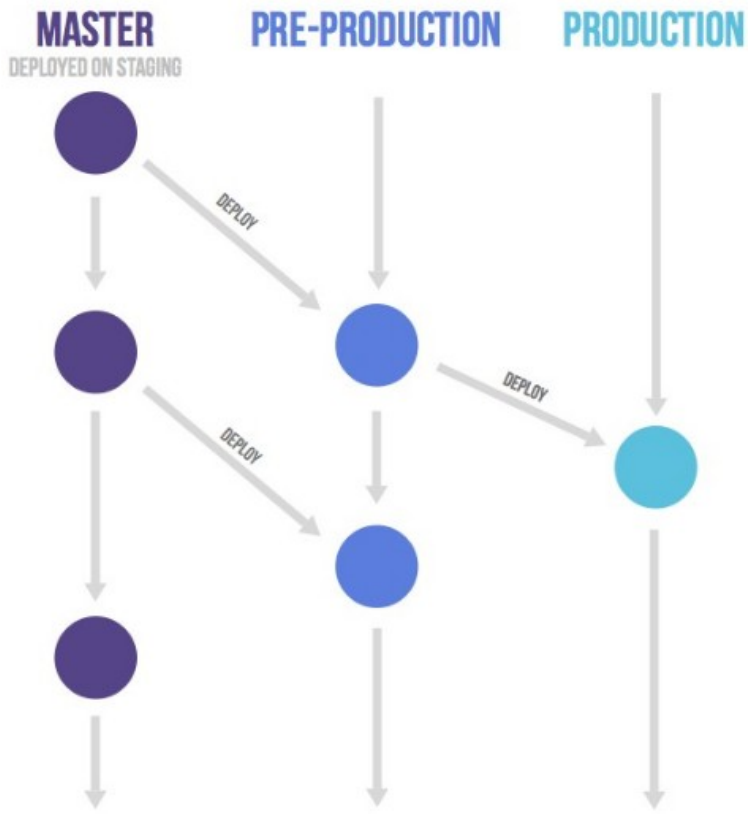
Configuration des MR

Dans le menu ***Project → Settings → General***, le mainteneur peut configurer les merge request

- Méthode de fusion :
  - Commit de merge
  - Commit de merge avec Rebasing si conflit
  - Rebasing obligatoire
- Options de fusion : Résolution automatique des discussions, hooks, Suppression de la branche source cochée par défaut
- Squash des commits (perte de l'historique des commits de la branche source)
  - Autoriser, Favoriser ou empêcher
- Vérifications avant la fusion
  - La pipeline doit s'être exécutée avec succès
  - Tous les discussions doivent être résolues

Autres workflow

- Il est possible de complexifier le workflow de collaboration autour de Gitlab.
- D'autres branches protégées représentant les différentes étapes de validation ou les différents environnements de déploiement peuvent être créées



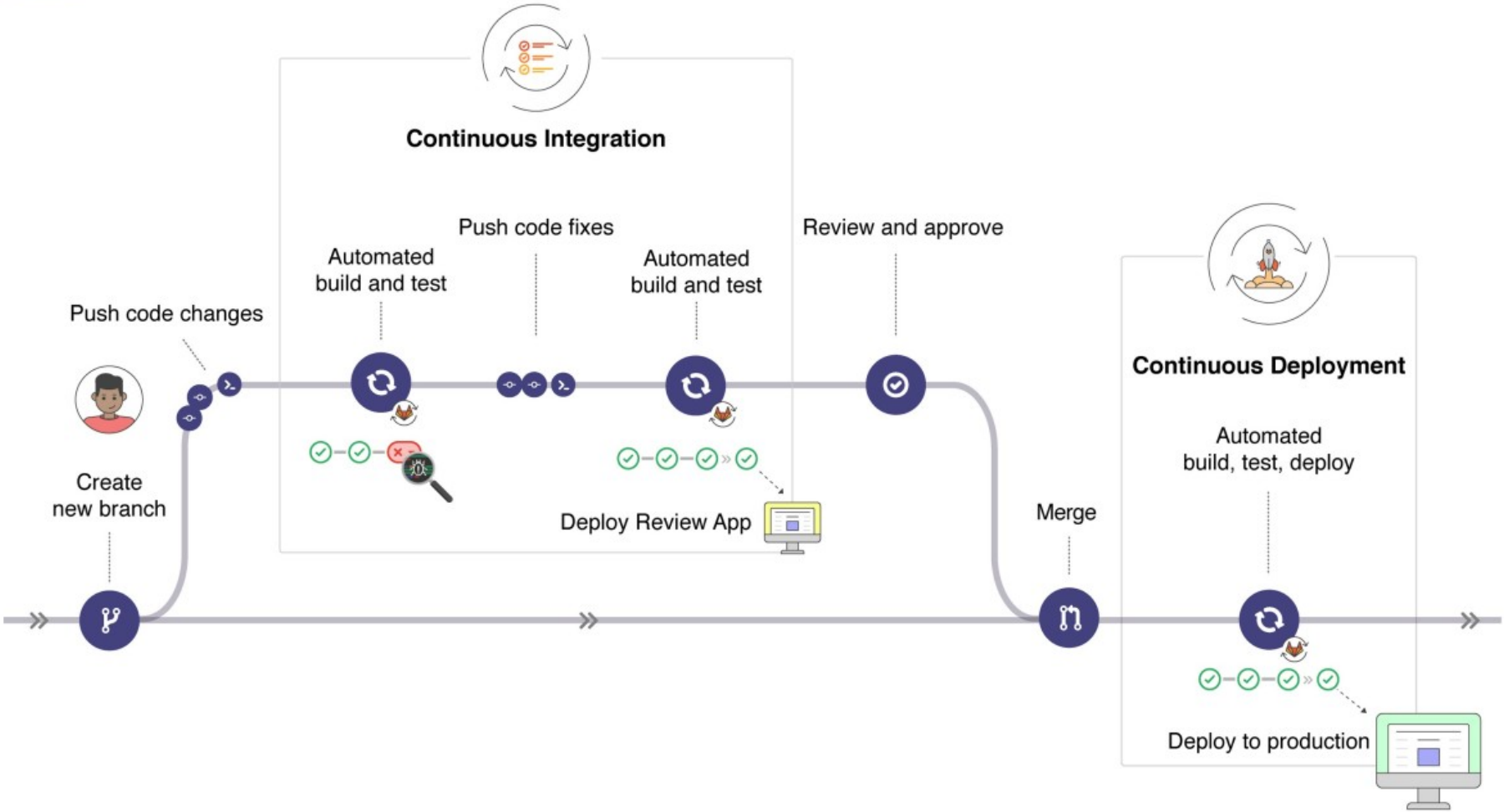
Bases pipeline Gitlab

Introduction
Jobs et Runners
UI Pipelines

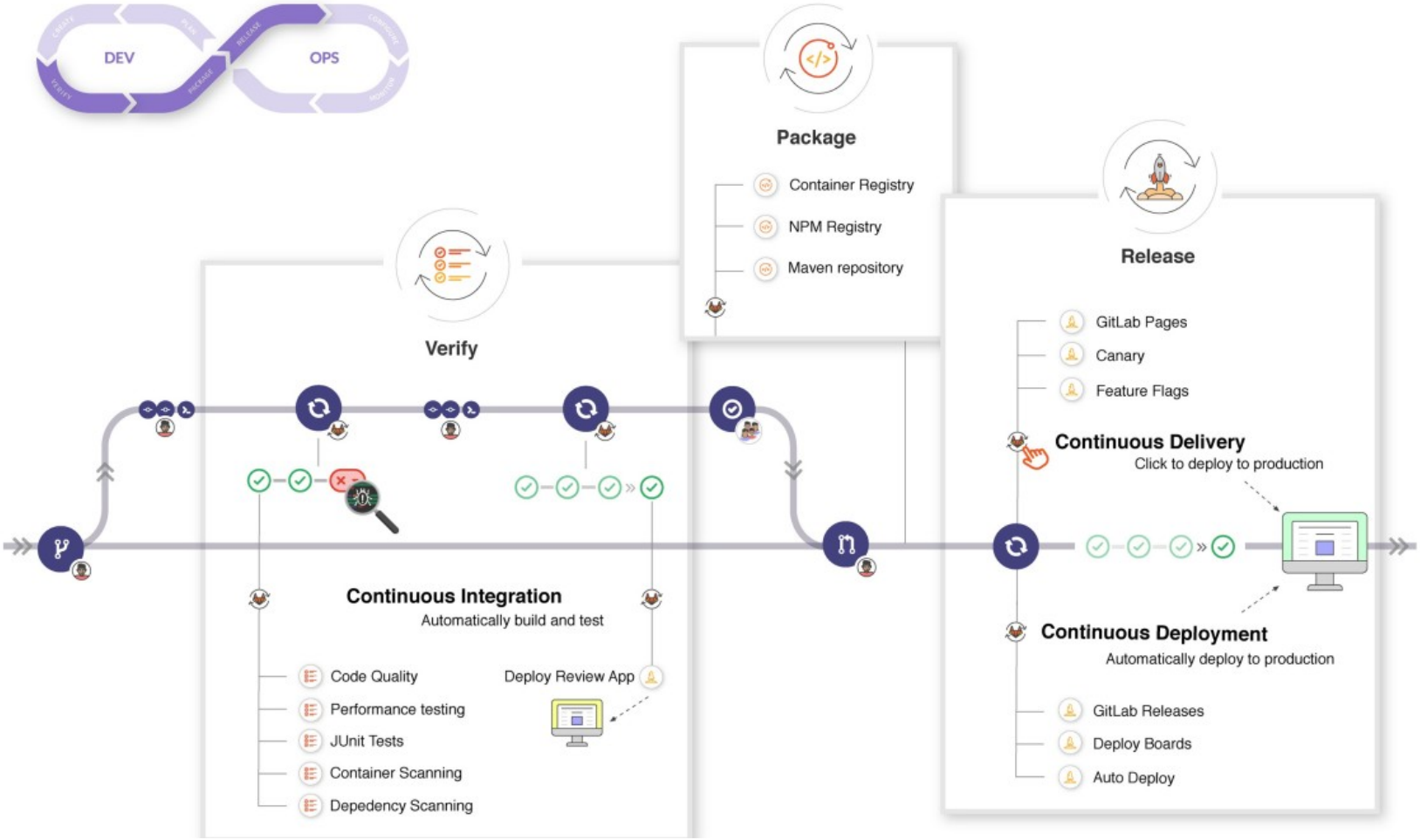
Introduction

- GitLab CI/CD est un outil permettant l’intégration, la livraison ou le déploiement continu
  - CI : A chaque push, une pipeline de scripts pour construire, tester, analyser est exécutée avant de la fusionner dans la branche principale
  - CD : A chaque push sur la branche principale, on effectue une release ou on déploie en prod
    Les dernières étapes peuvent être manuelles

CI/CD


Continuous Integration
Push code fixes
Review and approve
Automated build and test
Automated build and test
Push code changes
Continuous Deployment
Automated build, test, deploy
Create new branch
Deploy Review App
Merge
Deploy to production

En plus détaillé



Bases pipeline Gitlab

Introduction

Jobs et Runners

UI Pipelines

Runner

Les jobs de builds sont exécutés via des **runners**

GitLab Runner est une application qui s’exécute sur des machines distinctes et qui communique avec Gitlab.

Un runner peut être

- dédié à un projet à un groupe de projet.
  Il est défini par le mainteneur de Projet
- ou peut être partagé par tous les projets.
  Il est alors défini par l’administrateur

Type de Runners

Un runner peut être une machine virtuelle, une machine physique, un conteneur docker ou un pod dans un cluster Kubernetes.
Le type de runner conditionne les pipelines qu'il peut exécuter

GitLab et les Runners communiquent via une API
=> La machine du runner doit avoir un accès réseau au serveur Gitlab.

Pour disposer d'un runner :

- Il faut l'installer
- Puis l'enregistrer soit comme runner partagé (administrateur), soit comme runner dédié au projet

Installation GitlabRunner

L’installation s’effectue :

- Via des packages Debian/Ubuntu/CentOS/RedHat
- Exécutable MacOs ou Windows
- Comme service Docker
- Auto-scaling avec Docker-machine
- Via Kubernetes

Mise en place de runner

Pour mettre en place un runner, il faut d'abord le créer dans l'interface
Build – Runners – New Runner

Obtenir ensuite un token glrt-XXXX

Puis exécuter dans l'environnement du runner la commande
gitlab-runner register --token

Exécuteurs

Les exécuteurs d'un runner ont un influence sur les jobs que le runneur peut exécuter :

- ***shell*** : Le plus facile à installer mais toutes les dépendances du projet doivent être pré-installés sur le runner (git, npm, jdk, …)
- **Virtual Machine** : Nécessite Virtual Box ou Parallels. Les outils sont pré-installés sur la VM
- **Docker** : Permet d'exécuter des builds avec une image docker. D'autres services docker peuvent être démarrés pendant le build
- **Gitlab Runner Autoscaler avec Fleeting** : Idem docker + auto-scaling. Les exécuteurs de build sont créés à la demande
- **Kubernetes** : Utilisation d'un cluster Kubernetes. Via l'API, le runner créé des pods (machine de build + services)
- ***ssh*** : Peu recommandé, exécute le build via ssh sur une machine distante

Configuration runner

- La configuration d'un runner présente dans ***/etc/gitlab-runner/config.toml*** déterminent d'autres paramètres techniques du runner

```
concurrent = 5 // Nombre de jobs concurents
check_interval = 0
[session_server]
session_timeout = 1800
[[runners]]
name = "Another shell executeur"
url = "http://localhost"
token = "1NkCzKU1x_S6hz6VQ2Uu"
executor = "shell"
[runners.custom_build_dir]
[runners.cache]
[runners.cache.s3]
[runners.cache.gcs]
```

Affectation d’un runner et tags

- Lorsqu’une pipeline doit être exécutée, Gitlab affecte un runner pour le job.
  - Il choisit de préférence un runner dédié au projet
  - Chaque runner peut également avoir une liste de tags et une pipeline peut définir également des tags
    => Gitlab recherche alors le runner ayant les mêmes tags que le job
- Si Gitlab ne trouve pas de runner adapté, la pipeline de démarre pas (état stuck)

Bases pipeline Gitlab

Introduction
Jobs et Runners
UI Pipelines

Editeur

Un éditeur en ligne de ***.gitlab-ci.yml*** est disponible

Il permet une validation de la syntaxe

Repository → Files → .gitlab-ci.yml → Pipeline Editor

Des gabarits sont également disponibles pour la plupart des technologies

Repository → New File → Apply Template → .gitlab-ci.yml → <techno>

Exécution des pipelines

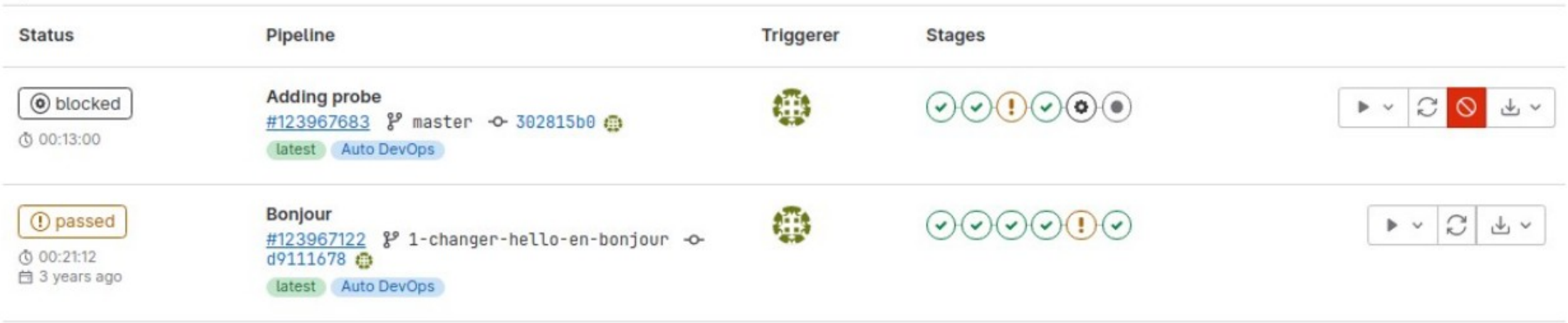
Les pipelines s’exécutent automatiquement à chaque push

Elles peuvent être également planifiées pour s’exécuter à des intervalles réguliers via l’UI ou l’API

Settings → CI/CD → Schedules

Enfin, elles peuvent être démarrées manuellement par l’UI

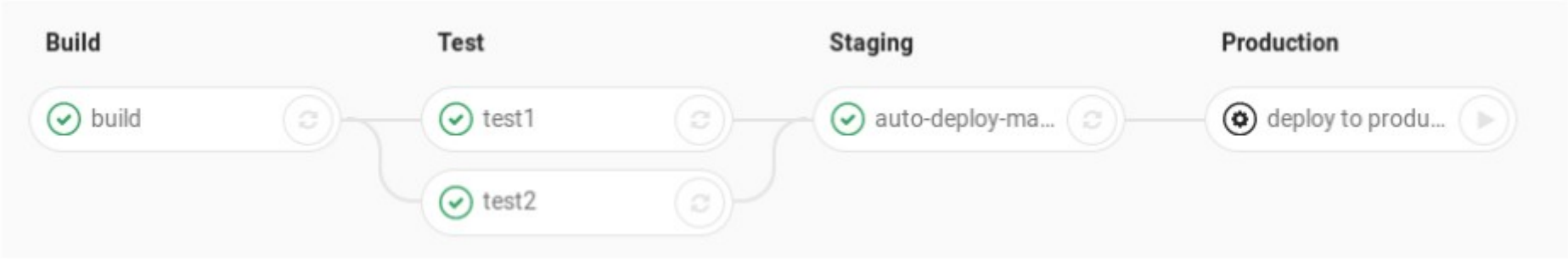
Tableau de bord d’exécution



Un tableau de bord permet de voir les dernières exécutions de pipeline, leurs statuts, le commit, le déclenchement et l’exécution des tâches.

Une barre de boutons permet de continuer ou redémarrer la pipeline et de télécharger les artefacts produits par le build.

Visualisation d’une pipeline



Le détail d’une pipeline est affichée graphiquement.

Il est possible de

- visualiser la sortie standard de chaque tâche
- déclencher manuellement une tâche

Atelier 3.2 : Utilisation de gabarit

Syntaxe gitlab-ci.yml

Basiques .gitlab-ci.yml
Principales Directives
Réutilisation
Intégration docker
Environnements et déploiements

Spécification de la pipeline

La spécification du job et de ses différentes phases peuvent être faits de différentes façons :

- ***AutoDevOps*** : Mode par défaut. Gitlab choisit la pipeline en fonction du projet. Nécessite des runners docker
- Fichier ***.gitlab-ci.yml*** à la racine du projet
  Des gabarits selon les piles technologies sont proposés par Gitlab

AutoDevOps

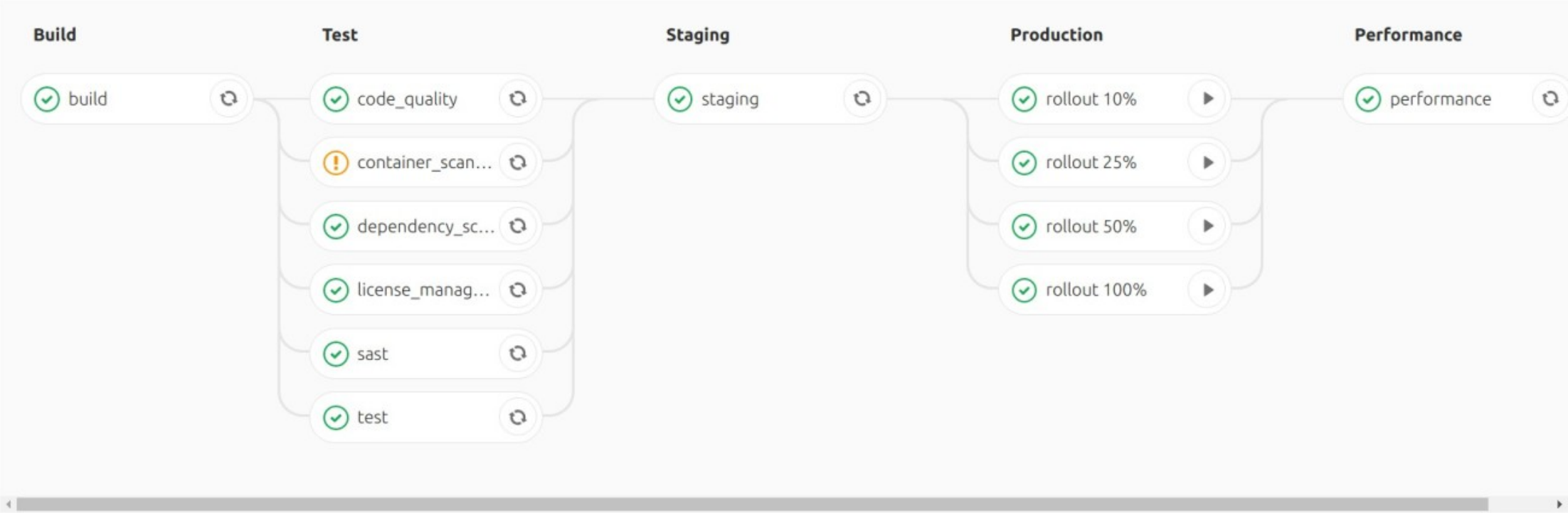
AutoDevOps est une pipeline adapté à toutes les technologies.

PréRequis : Docker pour builder, tester construire, Kubernetes pour déployer

Phases :

- Sur toutes les branches :
  - Build : Compilation, packaging
  - Test : Tests, Analyse qualité, Scan sécurité, licences
- Sur branche de feature
  - Review : Déploiement sur un environnement dédié à la branche
- Sur la branche par défaut
  - Staging : Déploiement sur Review App
  - Production : Roll-out manuel de la production
  - Performance : Test de performance en prod

AutoDevops sur branche main



Jobs / phases / tâches

Le fichier *.gitlab-ci.yml* défini des **jobs**.

Les jobs sont associés à des **phases** exécutées séquentiellement.

- Les jobs d'une même phase sont exécutées en parallèle
- Par défaut, si une phase échoue, les phases suivantes ne sont pas exécutées.

Un job est constitué d'une ou plusieurs commandes shell exécutées séquentiellement sur la machine de build (runner, image docker ou autre) .

Les jobs peuvent récupérer ou sauvegarder des résultats par le biais du serveur Gitlab

stages

La directive stages permet de définir les phases séquentielles de la pipeline

Elle se place dans la partie globale de .gitlab-ci.yml

- Si elle n'est pas présente, les phases par défaut sont : ***.pre, build, test, deploy, .post***
- Exemple :

```
stages:
- build
- test
- deploy
```

Jobs

Chaque jobs est défini par un nom et est associé à un stage. (Si le stage n'est pas précisé, le job appartient au stage test)

Les tâches exécutés par le job sont définies par les directives :

- ***script*** : Décrit les commandes du job
- ***before-script, after-script*** : Les commandes exécutées avant/après chaque job.

Exemple

```
stages:
  - Build
  - Test
  - Staging
  - Production

build:
  stage: Build
  script: make build dependencies

test1:
  stage: Test
  script: make build artifacts

test2:
  stage: Test
  script: make test

auto-deploy-ma:
  stage: Staging
  script: make deploy

deploy-to-production:
  stage: Production
  script: make deploy
```

Directive needs

Introduit dans Gitlab 12.2, la directive ***needs*** permet d'exprimer des dépendances entre jobs sans prendre en compte les stage.

Dans l'exemple suivant, le job *linux:rspec* s'exécute dés que le job *linux:build* est terminé (même si *mac:build* n'est pas terminé)

```
linux:build:
  stage: build
  script: echo "Building linux..."
mac:build:
  stage: build
  script: echo "Building mac…"
linux:rspec:
  stage: test
  needs: ["linux:build"]
  script: echo "Running rspec on linux..."
```

Contexte des directives

En fonction de leur niveau (indentation *yml*), les directives s'appliquent à l'ensemble des jobs ou à un job particulier.

Les principales directives globales sont :

- **default** : Valeurs par défauts des jobs. Inclut entre autres :
  - **image** : L'image docker utilisée pour le build (Nécessite un runner docker)
  - **services** : Les services devant être démarrés avant le build
  - **tag** : Tags du jobs permettant de l'affecter au bon runner
  - **timeout, retry, cache**, ...
- **variables** : Variables d'environnement disponibles pour le job

Variables

Le job peut accéder à un ensemble de variables :

- Fournies **systématiquement** par GitLab : Id d'issue, commit ID, branch ...
- Définies par **l'UI** au niveau transverse projet (administrateur), au niveau groupe ou au niveau projet.
- Définies dans ***.gitlab-ci.yml*** au niveau global ou job

Une variable peut être configurée comme étant masquée ou protégée[1]

L'accès se fait via la notation ***${variable}***

Ex :

```
docker login -u "$CI_REGISTRY_USER" -p
"$CI_REGISTRY_PASSWORD" $CI_REGISTRY
```

1. Accessible seulement des branches protégées

Syntaxe gitlab-ci.yml

Basiques .gitlab-ci.yml
Principales Directives
Réutilisation
Intégration docker
Environnements et déploiements

Artifacts

Les **artifacts** sont une liste de fichiers et répertoires attachés à un job terminé.

Ils sont téléchargeable (tar.gz) via l'UI

Ils sont conservés 1 semaine (par défaut)

Ils sont téléchargés par défaut par les jobs en aval

```
pdf:
  script: xelatex mycv.tex
  artifacts:
    paths:
    - mycv.pdf
    expire_in: 1 week
```

Réutilisation des artefacts

La directive ***dependencies*** permet de contrôler les artefacts que l'on veut récupérer

- Soit elle indique le nom des jobs dont on veut récupérer les artefacts
- Soit elle indique une liste vide pour indiquer que l'on ne veut pas récupérer les artefacts

Si, les dépendances ne sont pas disponibles lors de l'exécution du job, il échoue.

Exemple

```
build:osx:
  stage: build
  script: make build:osx
  artifacts:
    paths:
    - binaries/

build:linux:
  stage: build
  script: make build:linux
  artifacts:
  paths:
  - binaries/

test:osx:
  stage: test
  script: make test:osx
  dependencies:
  - build:osx
```

Cache des dépendances

- Le cache des dépendances permet d'accélérer l'exécution des jobs.
- Particulièrement utile lorsque le projet utilise de nombreuses librairies sur Internet
- Les caches sont désactivés si ils ne sont pas définis globalement ou par projet
  - Globalement, le cache est réutilisé entre jobs de différentes pipeline
  - Par projet, le cache est utilisé
    - par la prochaine pipeline, par le job l'ayant défini
    - Par un job aval qui défini un cache du même nom.
- Les caches sont gérés par les Runner (éventuellement téléchargé sur S3 si cache distribué).

Exemple

```
image: node:latest

# Cache modules in between jobs
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
  - node_modules/

before_script:
  - npm install

test_async:
  script:
  - node ./specs/start.js ./specs/async.spec.js
```


Atelier 4.2 : Pipeline avec artefact et dependencies

GIT_STRATEGY

La variable ***GIT_STRATEGY*** peut être positionnée dans la pipeline pour conditionner, l'interaction du runner avec le dépôt.

La variable peut prendre 3 valeurs :

- ***clone*** : Le dépôt est cloné par chaque job
- ***fetch*** : Réutilise le précédent workspace si il existe en se synchronisant ou effectue un clone
- ***none*** : N'effectue pas d'opération git, utilisé pour les taches de déploiement qui utilisent des artefacts précédemment construits

GIT_CHECKOUT

La variable ***GIT_CHECKOUT*** peut être utilisée lorsque GIT_STRATEGY est définie à *clone* ou *fetch*

Elle spécifie si une extraction git doit être exécutée (true défaut)

Si false :

- ***fetch*** : Met à jour le dépôt et laisse la copie de travail sur la révision courante ,
- ***clone*** : Clone le dépôt et laisse la copie de travail sur la branche par défaut

```
variables:
  GIT_STRATEGY: clone
  GIT_CHECKOUT: "false"
script:
  - git checkout -B main origin/main
  - git merge $CI_COMMIT_SHA
```

Control Flow

allow_failure permet à une tâche d'échouer sans impacter le reste de la pipeline.

- La valeur par défaut est false, sauf pour les jobs manuels.

retry permet de configurer le nombre de tentatives avant que le job soit en échec.

tags : Liste de tags pour sélectionner un runner

parallel : Nombre d'instances du jobs exécutés en parallèle

trigger : Permet de déclencher une autre pipeline à la fin d'un job.

Conditions

when conditionne l'exécution d'un job. Les valeurs possibles sont :

- ***on_success*** : Tous les jobs des phases précédentes ont réussi (défaut).
- ***on_failure*** : Au moins un des jobs précédents a échoué
- ***always*** : Tout le temps
- ***manual*** : Exécution manuelle déclenchée par l'interface

only et *except* limitent l'exécution d'un job à une branche ou une tag. Il est possible d'utiliser des expressions régulières (Déprécié !!)

rules

La directives **rules** permet d’inclure ou d’exclure des jobs de la pipeline.

- C’est la même finalité que only, except mais en plus puissant

Les règles sont évaluées lors de la création du pipeline dans l'ordre de définition

Lorsqu'une correspondance est trouvée, la tâche est incluse ou exclue du pipeline, selon la configuration.

Les règles sont définies avec les mots-clés ***if ,changes, exists, allow_failure, variables, when***

Exemple *rules:if*

```
job:
  script: echo "Hello, Rules!"
  rules:
  # On interdit pour une MR d'une branche feature
  # vers un autre branche que celle par défaut
  - if: $CI_MERGE_REQUEST_SOURCE_BRANCH_NAME =~ /^feature/
   &&
$CI_MERGE_REQUEST_TARGET_BRANCH_NAME != $CI_DEFAULT_BRANCH
    when: never
  # Si démarrage manuel, on autorise les échecs
  - if: $CI_MERGE_REQUEST_SOURCE_BRANCH_NAME =~ /^feature/
    when: manual
    allow_failure: true
  # Le job s'exécute si c'est une fusion de MR
  - if: $CI_MERGE_REQUEST_SOURCE_BRANCH_NAME
```

Exemple rules:variables

```
job:
  variables:
    DEPLOY_VARIABLE: "default-deploy"
  rules:
  - if: $CI_COMMIT_REF_NAME == $CI_DEFAULT_BRANCH
    variables:
      # Surcharge DEPLOY_VARIABLE
      DEPLOY_VARIABLE: "deploy-production"
  - if: $CI_COMMIT_REF_NAME =~ /feature/
    variables:
      # Définition nouvelle variable
      IS_A_FEATURE: "true"
  script:
  - echo "Run script with $DEPLOY_VARIABLE as an argument"
  - echo "Run another script if $IS_A_FEATURE exists"
```

workflow

La directive globale **workflow** permet de contrôler l'exécution de la pipeline complète.

- ***auto_cancel*** permet d'annuler une pipeline en cours d'exécution
- ***on_new_commit*** : Annulation si un nouveau commit survient
- ***on_job_failure*** : Nécessite une configuration de l'administrateur, permet d'annuler certains jobs de la pipeline lorsqu'un job est en échec
- ***name*** permet de nommer la pipeline
- ***rules*** permet de conditionner l'exécution en fonction des variables prédéfinies fournies par Gitlab

Exemples

```
# Annule les jobs ayant la propriété interruptible à true
# si un nouveau commit survient
workflow:
  auto_cancel:
    on_new_commit: interruptible
job1:
  interruptible: true
  script: sleep 60
job2:
  interruptible: false
  script: sleep 60
```

Syntaxe gitlab-ci.yml

Basiques .gitlab-ci.yml
Principales Directives
Réutilisation
Intégration docker
Environnements et déploiements

Inclusion

Le mot-clé ***include*** permet l'inclusion de fichiers YAML externes.

4 méthodes d’inclusions :

- ***local*** : Inclusion d’un fichier du dépôt
- ***file*** : Inclusion du fichier d’un autre projet
- ***template*** : Inclusion d’un template fourni par Gitlab. Le gabarit peut être surchargé
- ***remote*** : Inclusion d’un fichier accessible via URL

Exemples

```
include:
  - remote: 'https://gitlab.com/awesome-project/raw/main/.before-script-template.yml'
  - local: '/templates/.after-script-template.yml'
  - template: Auto-DevOps.gitlab-ci.yml
  - project: 'my-group/my-project'
    ref: main
    file: '/templates/.gitlab-ci-template.yml
```

Surcharge de gabarit

Gabarit :

```
variables:
  POSTGRES_USER: user
  POSTGRES_PASSWORD: testing_password
  POSTGRES_DB: $CI_ENVIRONMENT_SLUG
production:
  stage: production
  script:
  - install_dependencies
  - deploy
  environment:
    name: production
    url: https://$CI_PROJECT_PATH_SLUG.
$KUBE_INGRESS_BASE_DOMAIN
    only:
    - main
```

- **Surcharge** :

```
include: 'https://company.com/autodevops-
template.yml'

image: alpine:latest

variables:
  POSTGRES_USER: root
  POSTGRES_PASSWORD: secure_password

stages:
  - build
  - test
  - production
production:
  environment:url: https://domain.com
```

Extension

Le mot réservé ***extends*** permet à un job d'hériter d'un autre (ou plusieurs)

Le job peut surcharger des valeurs du parent. Ex :

```
tests:
  script: rake test
  stage: test
  only:
    refs:
    - branches

rspec:
  extends: .tests
  script: rake rspec
  only:
    variables:
    - $RSPEC
```

Syntaxe gitlab-ci.yml

Basiques .gitlab-ci.yml
Principales Directives
Réutilisation
Intégration docker
Environnements et déploiements

Docker

Le mot-clé ***image*** spécifie l'image docker à utiliser pour exécuter la pipeline.

Il peut être global à la pipeline (directive default) ou spécifique à un job.

Par défaut, l'exécuteur utilise Docker Hub mais cela peut être configuré via *gitlab-runner/config.toml*

Syntaxe image

- 2 syntaxes sont possibles pour image
  - Si juste à spécifier le nom de l'image :
    ```
    image: "registry.example.com/my/image:latest"
    ```
  - Si l'on veut passer d'autres options, il faut utiliser la clé name
    ```
    image:
      name: "registry.example.com/my/image:latest"
      entrypoint: ["/bin/bash"]
      pull-policy : if-not-present
      docker :
        platform: arm64/v8
        user: dave
    ```
- La clé entrypoint équivalent à l'argument --entrypoint de la commande docker
- La clé docker permet de passer des options à l'exécuteur docker

Docker services

Le mot-clé ***services*** permet de démarrer d'autres container durant le build.

Le build peut alors accéder au service via le nom de l'image (ou un alias)

```
services:
  - tutum/wordpress:latest
  alias : wordpress
```

Le service est accessible via tutum-wordpress, tutum/wordpress, wordpress

Test du service

Lors de l’exécution du build, le Runner:

- Vérifie quels ports sont ouverts
- Démarre un autre conteneur qui attend que ces ports soient accessibles

Si ces tests échouent, un message apparaît dans la console :

```
*** WARNING: Service XYZ probably didn't start properly.
```

Options pour service

4 options disponibles :

- ***name*** : Nom de l'image.
  Requis si l'on veut passer d'autres options
- ***entrypoint*** : L'argument --entrypoint de docker.
  Syntaxe équivalente à la directive ENTRYPOINT de docker
- ***command*** : Passer en argument de la commande docker.
  Syntaxe équivalente à la directive CMD de docker
- ***alias*** : Un alias d'accès dans le DNS

Variables

Les variables définies dans le fichier YAML sont fournies au conteneur exécutant le service.

Exemple service Postgres :

```
services:
  - postgres:latest
variables:
  POSTGRES_DB: nice_marmot
  POSTGRES_USER: runner
  POSTGRES_PASSWORD: ""
```

Construction d’image

Un scénario désormais classique du CI/CD est:

1) Créer une image applicative

2) Exécuter des tests sur cette image

3) Pousser l’image vers un registre distant

4) Déployer l’image vers un serveur

En commande docker :

```
docker build -t my-image dockerfiles/
docker run my-image /script/to/run/tests
docker tag my-image my-registry:5000/my-image
docker push my-registry:5000/my-image
```

Configuration du runner

3 possibilités afin de permettre l'exécution de commande docker :

- Avec l'exécuteur shell et une pré-installation de docker sur le runner
- Avec l'exécuteur docker et :
  - l'image docker (image contenant le client docker),
  - ainsi que le service docker-in-docker permettant de disposer d'un daemon docker
- Avec l'exécuteur docker, une pré-installation de docker sur le runner et une redirection de socket docker pour profiter du démon installé

Attention : Pour ces 3 techniques le serveur gitlab doit être accessible des containers

Exécuteur shell

1. Enregistrer un exécuteur Shell sur le runner :

```
sudo gitlab-runner register -n \
--url https://gitlab.com/ \
--registration-token REGISTRATION_TOKEN \
--executor shell \
--description "My Runner"
```

2. Installer docker sur la machine hébergeant le runner

3. Ajouter l'utilisateur gitlab-runner au groupe docker

```
sudo usermod -aG docker gitlab-runner
```

4. Vérifier que gitlab-runner a accès à docker

```
sudo -u gitlab-runner -H docker info
```

5. Tester la pipeline :

```
before_script:
- docker info
build_image:
script:
- docker build -t my-docker-image .
- docker run my-docker-image /script/to/run/tests
```

Docker in Docker (1)

Enregistrer un exécuteur docker en mode privilège

```
sudo gitlab-runner register -n \
--url https://gitlab.com/ \
--registration-token REGISTRATION_TOKEN \
--executor docker \
--description "My Docker Runner" \
--docker-image "docker:stable" \
--docker-privileged
```

Docker in Docker (2)

Tester dans un .gitlab-ci.yml

```
image: docker:stable
variables:
  DOCKER_HOST: tcp://docker:2375/
  DOCKER_DRIVER: overlay2
services:
  - docker:dind
before_script:
  - docker info
...
```

Association de socket (1)

- Enregistrer un runner avec une association de socket :

```
sudo gitlab-runner register -n \
--url https://gitlab.com/ \
--registration-token REGISTRATION_TOKEN \
--executor docker \
--description "My Docker Runner" \
--docker-image "docker:stable" \
--docker-volumes
/var/run/docker.sock:/var/run/docker.sock
```

Association de socket (2)

```
image: docker:stable
before_script:
  - docker info
build:
  stage: build
  script:
  - docker build -t my-docker-image .
  - docker run my-docker-image
/script/to/run/tests
```

Registre Gitlab

Une fois l'image construite, il est naturel de la pousser dans un registre

Gitlab dans sa version entreprise propose un registre de conteneur.

Pour l'utiliser, il faut :

- Que l'administrateur est autorisé le registre Docker
  Nécessite un nom de domaine
- De s'authentifier auprès du registre.
- Utiliser docker build --pull pour récupérer les changements sur l'image de base
- Faire explicitement un docker pull avant chaque docker run. Sinon, on peut être gêné par des problèmes de cache si l'on a plusieurs runner.
- Ne pas construire directement vers le tag latest si plusieurs jobs peuvent être lancés simultanément

Authentification auprès du registre Gitlab

Si le registre hébergé par Gitlab est autorisé, 3 façons sont disponibles pour l'authentification :

- Utiliser les variables $CI_REGISTRY_USER et $CI_REGISTRY_PASSWORD qui sont des crédentiels éphémères disponibles pour le job
- Utiliser un jeton d'accès personnel ***User Settings → Access token***
- Utiliser le jeton de déploiement : *gitlab-deploy-token*

Exemple

```
build:
  image: docker:27
  services:
    - docker:dind
  variables:
    DOCKER_HOST: tcp://docker:2376
    DOCKER_DRIVER: overlay2
  stage: build
  script:
  - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  - docker build -t $CI_REGISTRY/group/project/image:latest .
  - docker push $CI_REGISTRY/group/project/image:lates
```

Syntaxe gitlab-ci.yml

Basiques .gitlab-ci.yml
Principales Directives
Réutilisation
Intégration docker
Environnements et déploiements

Introduction

GitLab CI/CD est également capable de deployer sur différents environnements

Les **environnements** sont comme des tags décrivant où le code est déployé

Les **déploiements** sont créés lorsque les job déploient des versions de code vers des environnement
=> ainsi chaque environnement peut avoir plusieurs déploiements

GitLab:

- Fournit un historique complet des déploiements pour chaque environnement
- Garde une trace des déploiements => On sait ce qui est déployé sur les serveurs

Définition des environnements

Les environnements sont définis dans .gitlab-ci.yml

Le mot-clé **environment** indique à GitLab que ce job est un job de déploiement. Il peut être associée à une URL

=> Chaque fois que le job réussit, un déploiement est enregistré, stockant le SHA Git et le nom de l'environnement.
Operate → Environments

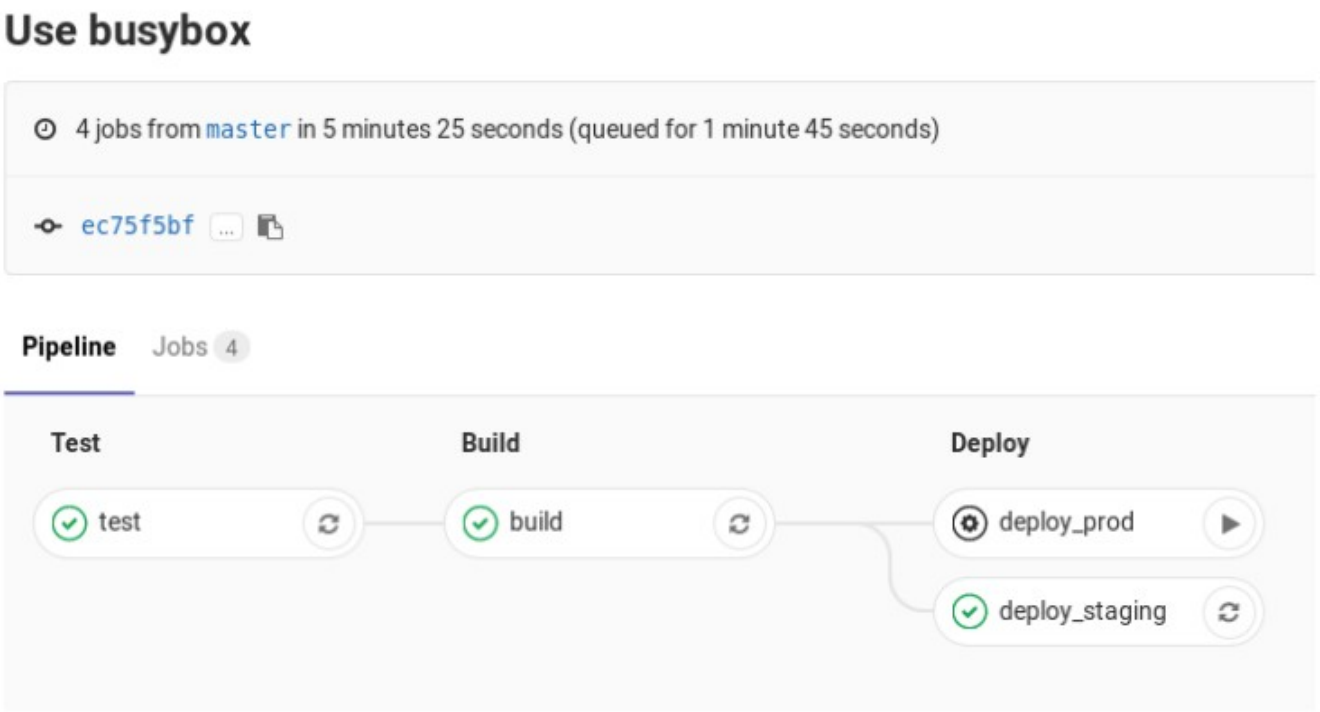
Le nom de l'environnement est accessible via le job par la variable $CI_ENVIRONMENT_NAME

Exemple

```
deploy_staging:
  stage: deploy
  script:
    - echo "Deploy to staging server"
  environment:
    name: staging
    url: https://staging.example.com
  only:
  - main
```

Déploiement manuel

L’ajout de when: manual convertit le job en un job manuel et expose un bouton Play dans l’UI



Environnements dynamiques

Il est possible de déclarer des noms d'environnement à partir de variables : **environnements dynamiques**

Les paramètres name et url peuvent alors utiliser :

- Les variables d'environnement prédéfinies
- Les variables de projets ou de groupes
- Les variables de .gitlab-ci.yml

Ils ne peuvent pas utiliser :

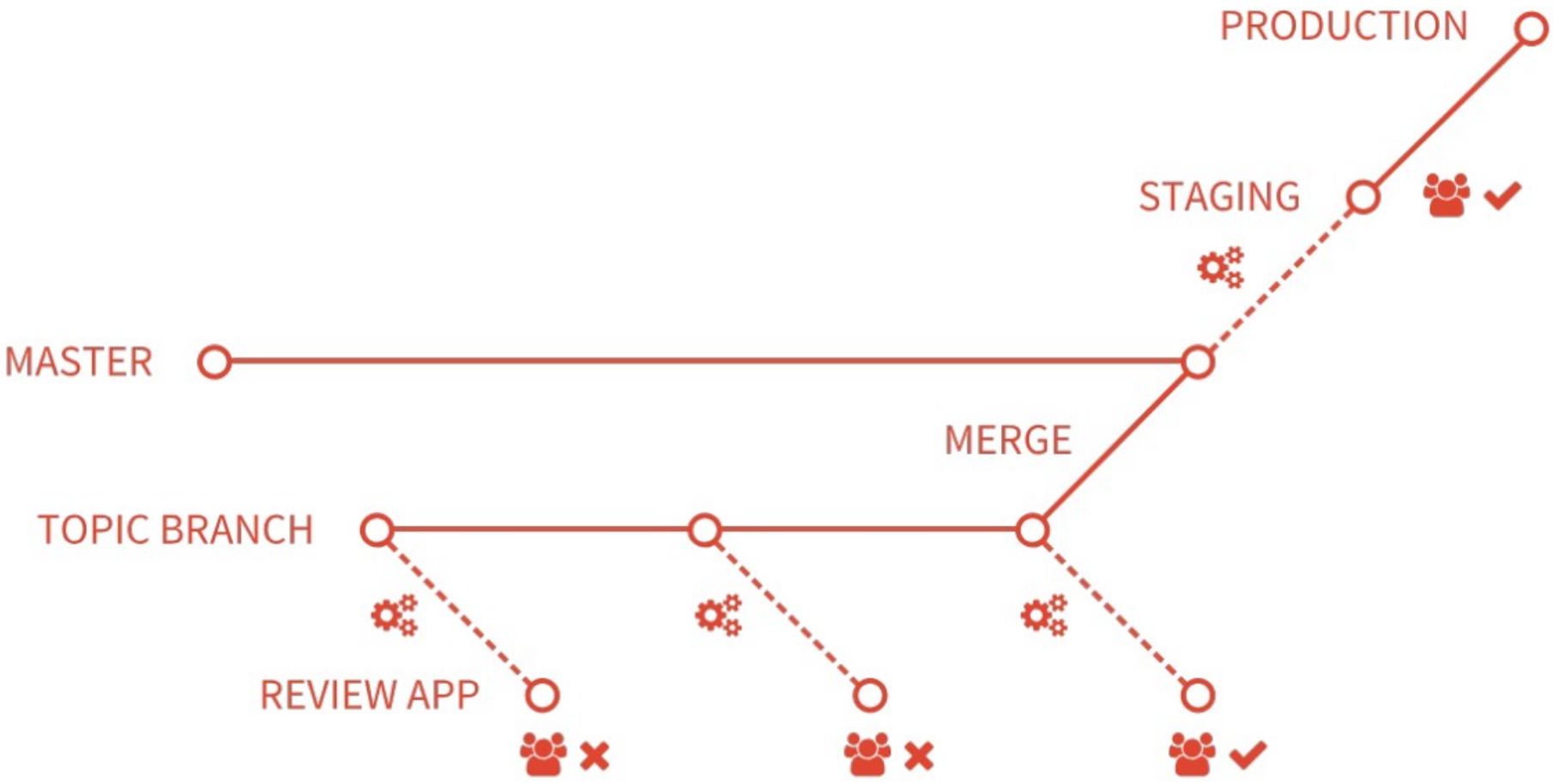
- Les variables définies dans script
- Du côté du runner

=> Il est possible de créer un environnement/déploiement pour chaque issue ou MR : Les Review Apps

Exemple

```
deploy_review:
  stage: deploy
  script:
    - echo "Deploy a review app"
  environment:
    name: review/$CI_COMMIT_REF_NAME
    url: https://$CI_ENVIRONMENT_SLUG.example.com
  only:
    - branches
  except:
    - main
```

Review App dans le workflow



Exemple complet

```
stages:
..
- deploy
...
deploy_review:
  stage: deploy
  script: echo "Deploy a review app"
  environment:
    name: review/$CI_COMMIT_REF_NAME
    url: https://$CI_ENVIRONMENT_SLUG.example.com
  only:
    - branches
  except:
    - main
deploy_staging:
  stage: deploy
  script: echo "Deploy to staging server"
  environment:
    name: staging
    url: https://staging.example.com
  only:
    - main
deploy_prod:
  stage: deploy
  script: echo "Deploy to production server"
  environment:
    name: production
    url: https://example.com
  when: manual
  only:
    - main
```

Arrêter un environnement

Arrêter un environnement consiste à appeler l'action ***on_stop*** si elle est définie.

Cela peut se faire par l'UI ou automatiquement dans la pipeline.

Lors du workflow Review App, l'action *on_stop* est automatiquement appelée à la suppression de la branche de feature.

Exemple

```
deploy_review:
  stage: deploy
  script:
  - echo "Deploy a review app"
  environment:
    name: review/$CI_COMMIT_REF_NAME
    url: https://$CI_ENVIRONMENT_SLUG.example.com
    on_stop: stop_review
  only:
    - branches
  except:
    - main

stop_review:
  stage: deploy
  variables:
    GIT_STRATEGY: none
  script:
    - echo "Remove review app"
  when: manual
  environment:
    name: review/$CI_COMMIT_REF_NAME
    action: stop
```

Support Gitlab pour pipeline standard

Construction et tests développeur
Analyses statiques
Dépôts d’artefacts
Release
Gestion de l’infrastructure
AutoDevOps

Construction

- Les premières tâches d’une pipeline consiste généralement
  - A exécuter les tests unitaires développeurs
  - Si ceux-ci réussissent, packager le code source
- En fonction des piles technologiques, le packaging du code source diffère
  - Java : Compilation + création d’un jar
  - Javascript : Minification et obfuscation du code + création d’un zip
  - –…
- Gitlab s’appuie alors sur des outils de la pile technologique qui sont soit préinstallés sur des runners ou dans des images docker

Outils de buid

La phase de build s'appuie typiquement sur un outil de build propre à la techno :

- ***Maven (Java)****:* Le plus répandu et le plus supporté.
- ***Gradle : « Build As Code »****.* S'applique à d'autres langages que Java (C, C+, Python, Php, …)
- ***npm, yarn, webpack****, … :* Monde JavaScript
- ***tslint*** *:* Linter typescript
- ***Composer, PhpUnit*** *:* PHP
- ***pip, unittest*** *:* Python

Support Gitlab pour les tests

Pour ces tests de début de pipeline, Gitlab permet :

- De publier les résultats des tests et de les attacher à une MR
- Si Ruby, de définir des FailFast Test permettant d’économiser des ressources runners

Publier les résultats de test

Gitlab ne supporte que le format JUnit

Il suffit de placer la directive ***artifacts:reports:junit*** dans .gitlab-ci.yml

```
ruby:
  stage: test
  script:
  - bundle exec rspec --format progress --format RspecJunitFormatter –out
   rspec.xml
  artifacts:
  when: always
  paths:
  - rspec.xml
  reports:
    junit: rspec.xml
```

Publication résultat de test

Gitlab propose alors des rapports de tests qui permettent de facilement isoler les tests ayant échoués

Pipeline | Needs | Jobs 1 | Failed Jobs 1 | **Tests 3**

Summary

3 tests | 2 failures | 0 errors | 33.33% success rate | 16.00ms

Jobs

Job	Duration	Failed	Errors	Skipped	Passed	Total
jest	16.00ms	2	0	0	1	3

Test summary contained 3 failed out of 3 total tests — View full report — Collapse

rspec found 1 failed out of 1 total test

× Failed 1 time in master in the last 14 days — User#full_name returns first_name + last_name

jest found 2 failed out of 2 total tests

× New — Calculator #add returns the sum of the 2 given numbers

× Failed 1 time in master in the last 14 days — Calculator #subtract returns the difference between the 2 given numbers

Atelier 5.1 : Publication test

Couverture des tests

Si l’on s’est équipé d’un outil calculant la couverture des tests, il est possible de publier ces métriques dans Gitlab.

Les résultats sont visibles :

- Dans les Merge request
- Dans les analytiques projets
- Dans les analytiques groupe
- Sous forme de badge au niveau d’un dépôt

Il est possible de conditionner une MR à un certain seuil de couverture **Premium**

Mise en place

La mise en place consiste à utiliser le mot-clé ***coverage*** dans .gitlab-ci.yml et d'indiquer une expression régulière permettant d'extraire l'information de la sortie standard.

Exemple JacoCo (Java/Kotlin)

`/Total.*?([0-9]{1,3})%/.`

Publication

Pipeline #4637061 passed with warnings for 99f87a83. View details

Coverage 91.98%

Accept Merge Request

The source branch will be removed.

Modify commit message

Post Test

passed | #5199993 | coverage | 01:20 | about 2 hours ago | 91.98%

Code coverage statistics for master Mar 12 - Jun 10

Download raw data (.csv)

rspec

Support Gitlab pour pipeline standard

Construction et tests développeur
Analyses statiques
Dépôts d’artefacts
Release
Gestion de l’infrastructure
AutoDevOps

Introduction

Gitlab fournit également du support pour les analyses statiques de code source

- Analyse qualité
- Utilisation des licences
- Détection de vulnérabilités

La mise en place s’effectue dans .gitlab-ci.yml et les résultats sont publiés dans le projet

Analyse qualité

Gitlab s'appuie sur des outils tierces, Sonarqube, ESLint, ...

Il faut alors

- appeler l'analyse à une étape de la pipeline
- Configurer l'outil pour qu'il génère un fichier de sortie au bon format
- Déclarer un artefact ***codequality***

Les résultats sont disponibles[1] :

  - Dans une Merge request
  - Dans le détail d'une pipeline
  - Dans la vue qualité d'un projet

Ils peuvent également être téléchargés au format brut

1. Dépend fortement de la licence

Exemple SonarQube

```
sonarqube-check:
  stage: sonarqube
  image:
    name: sonarsource/sonar-scanner-cli:latest
    entrypoint: [""]
  variables:
    SONAR_USER_HOME: "${CI_PROJECT_DIR}/.sonar"
    GIT_DEPTH: "0"
  script:
    # Analyse SonarQube
    - sonar-scanner
        -Dsonar.projectKey=${CI_PROJECT_PATH_SLUG}
        -Dsonar.sources=.

    # Conversion du rapport Sonar → format Code Climate (JSON)
    - |
      curl -s "${SONAR_HOST_URL}/api/issues/search\
        ?componentKeys=${CI_PROJECT_PATH_SLUG}\
        &resolved=false&ps=500\
        &additionalFields=rules" \
        -H "Authorization: Bearer ${SONAR_TOKEN}" \
        | python3 sonar_to_codeclimate.py > gl-code-quality-report.json
  artifacts:
    reports:
      codequality: gl-code-quality-report.json  # ← directive clé GitLab
    expire_in: 1 week
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
  allow_failure: true
```

Affichage Merge Request

Code quality degraded on 7746 points

Collapse

- Critical - Consider simplifying this complex logical expression.
  in scripts/frontend/stylelint/stylelint-utils.js:15
- Critical - Consider simplifying this complex logical expression.
  in app/assets/javascripts/projects/settings/access_dropdown.js:248
- Critical - CSRF vulnerability in OmniAuth's request phase
  in Gemfile.lock:810
- Major - Function `buildConfig` has 7 return statements (exceeds 4 allowed).
  in workhorse/main.go:67
- Major - Function `run` has 8 return statements (exceeds 4 allowed).
  in workhorse/main.go:152
- Major - Method `Resizer.Inject` has 5 return statements (exceeds 4 allowed).
  in workhorse/internal/imageresizer/image_resizer.go:164
- Major - Method `Resizer.tryResizeImage` has 7 return statements (exceeds 4 allowed).
  in workhorse/internal/imageresizer/image_resizer.go:268
- Major - Function `unpackFileFromZip` has 7 return statements (exceeds 4 allowed).
  in workhorse/internal/artifacts/entry.go:64

Sécurité

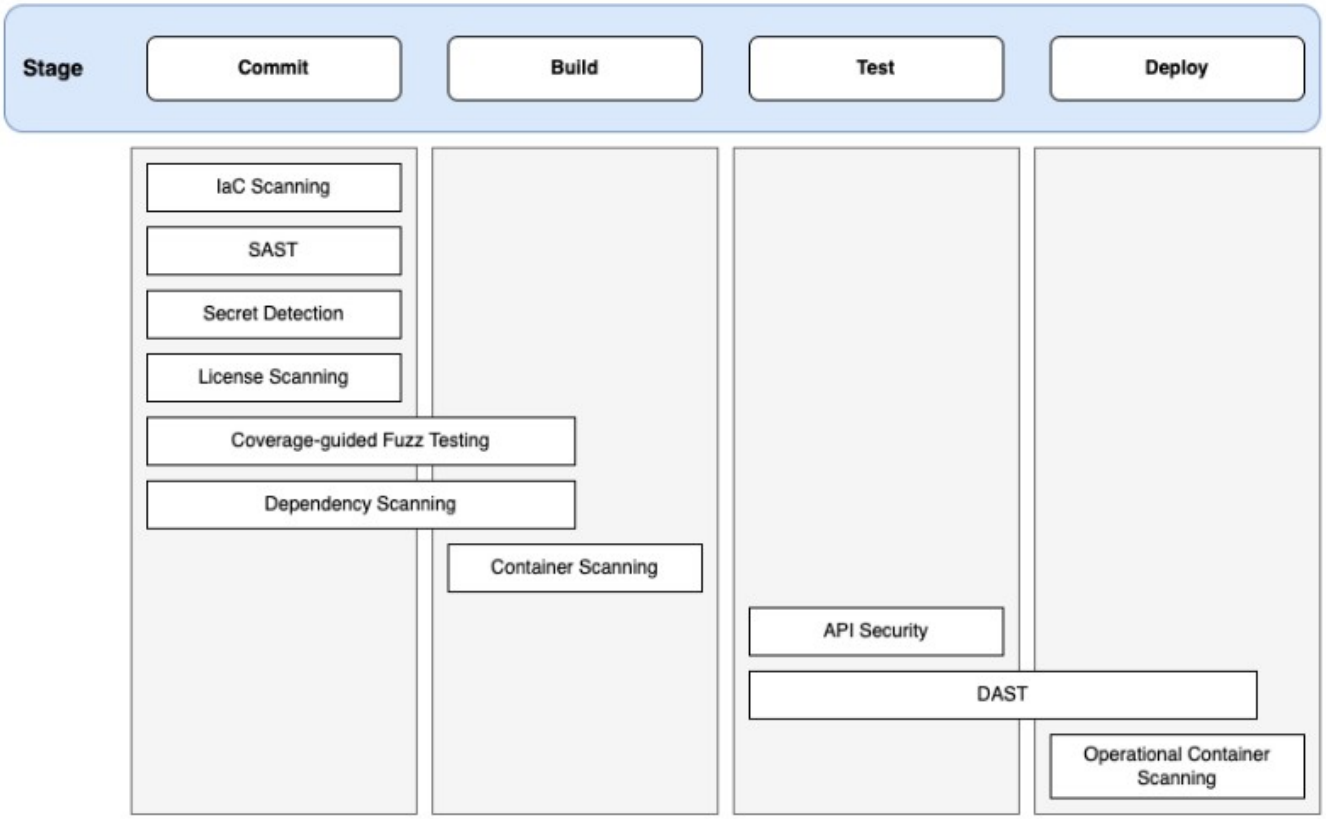
GitLab analyse la sécurité d’une application, soit dans le cadre d’une pipeline, soit de façon planifiée.

Cela couvre :

- Le **code source**
  Static Application Security Testing (SAST) + Analyze du dépôt pour la détection de secrets
- Les **dépendances** (Librairies, conteneurs)
  Dependencies Scanning, Container Scanning
- Les **vulnérabilités** dans une application Web en cours d'exécution.
  Dynamic Application Security Testing (DAST), Analyse des APIs pour détection de vulnérabilités connus ou inconnues (API fuzzing)
- **L’infrastructure** : Configuration de l'IAC (Infrastructure As Code)

Étapes vs Outils

Chaque outil intervient à différentes étapes de la pipeline



Mise en place

La mise en place de ces outils dépend fortement de la licence et certains outils ne sont disponibles qu’en version payante.

Les résultats des outils peuvent être publiés dans l’interface si une directive ***artifacts:reports <mot-clé>*** est présente dans *.gitlab-ci.yml*

Licences

Gitlab permet également de fixer des politiques transverses quant à l’utilisation de produit tiers.

L’analyse de code permet de détecter les licences associées aux dépendances et d’afficher des rapports si le projet utilise des dépendances non-permises.

Outils Free

SAST : Static Application Security Testing

```
stages:
- test
sast:
  stage: test
include:
- template: Security/SAST.gitlab-ci.yml
```

Secret Detection

```
include:
- template: Security/Secret-Detection.gitlab-ci.yml
```

Infrastructure as Code (IaC) Scanning

```
include:
- template: Security/Secret-Detection.gitlab-ci.yml
```

Support Gitlab pour pipeline standard

Construction et tests développeur
Analyses statiques
Dépôts d’artefacts
Release
Gestion de l’infrastructure
AutoDevOps

Gitlab

Gitlab offre plusieurs supports pour stocker et partager les artefacts construits :

- **GitLab Package Registry** est un registre privé ou public supportant les gestionnaires de packages courants : Composer, Conan, Generic, Maven, npm, NuGet, PyPI, RubyGem
- **GitLab Container Registry** est un registre privé pour les images Docker
- **GitLab Terraform Module Registry** supporte les modules Terraform

D'autre part, *Dependency Proxy* est un proxy local utilisé pour les images et paquets fréquemment utilisés.

Bien sûr, il est possible d'intégrer des solutions tierces (Nexus, Artifactory ...)

Gitlab Package Registry

GitLab Package Registry est en fait un projet Gitlab sans dépôt de source associé.

Les packages publiés héritent de la visibilité du projet

Pour y publier, les autres projets doivent s’authentifier auprès du registre via des jetons (personnel ou job) et configurer leur outil de build afin de publier vers la bonne URL

Exemple Maven

settings.xml

```
<server>
  <id>gitlab-maven</id>
  <configuration>
    <httpHeaders>
      <property>
        <name>Job-Token</name>
        <value>${env.CI_JOB_TOKEN}</value>
      </property>
    </httpHeaders>
  </configuration>
</server>
```

pom.xml

```
<distributionManagement>
  <repository>
    <id>gitlab-maven</id>
    <url>https://gitlab.example.com/api/v4/projects/<project_id>/packages/maven</url>
  </repository>
  <snapshotRepository>
    <id>gitlab-maven</id>
    <url>https://gitlab.example.com/api/v4/projects/<project_id>/packages/maven</url>
  </snapshotRepository>
</distributionManagement>
```

GitLab Container Registry

Avec ***GitLab Container Registry***, chaque projet peut avoir son propre espace pour stocker les images Docker.

La fonctionnalité doit être activée par l'administrateur.

Elle n'est typiquement accessible qu'en ***https***

Les images doivent suivre une convention de nommage :

<registry URL>/<namespace>/<project>/<image>

Des permissions fines peuvent être associés au registre

Support Gitlab pour pipeline standard

Construction et tests développeur
Analyses statiques
Dépôts d’artefacts
Release
Gestion de l’infrastructure
AutoDevOps

Release

Dans GitLab, une ***Release*** permet de créer un instantané du projet incluant les packages et les notes de version.

- La release peut être créée sur n'importe quelle branche.
- La création d'une Release crée un tag.
  Si le tag est supprimé, la release également.

Contenu et création d’une release

Une release peut contenir :

- Un instantané du code source
- Des packages créés à partir des artefacts des jobs
- Des méta-données de version
- Des releases notes

Création : Edition

Une release peut être créée

- Via un Job CI/CD
- Manuellement via l’UI Releases page
- Via l’API

Après avoir créé une release, on peut :

- Ajouter des release notes
- Ajouter un message pour le tag associé
- Associer des milestones avec
- Joindre des ressources (packages ou autres)

Création de release via un job CI/CD

Dans .gitlab-ci.yml, on crée des release en utilisant le mot-clé ***release***

3 méthodes typiques :

- Créer une release lorsqu'un tag est créé
- Créer une release quand un commit est fusionné dans la branche par défaut.
- Créer les méta-données de release dans un script personnalisé.

Exemple

Création lors fusion dans la branche par défaut

```
release_job:
  stage: release
  image: registry.gitlab.com/gitlab-org/release-cli:latest
  rules:
  - if: $CI_COMMIT_TAG
    when: never # Ne pas exécuter si création manuelle de tag
  - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH # Branche par défaut
  script:
  - echo "running release_job for $TAG"
  release:
    tag_name: 'v0.$CI_PIPELINE_IID' # Version incrémentée par pipeline
    description: 'v0.$CI_PIPELINE_IID'
    ref: '$CI_COMMIT_SHA' # tag créé à partir du SHA1.
```

Support Gitlab pour pipeline standard

Construction et tests développeur
Analyses statiques
Dépôts d’artefacts
Release
Gestion de l’infrastructure
AutoDevOps

Introduction

La gestion de l’infrastructure avec Gitlab s’appuie sur ***Terraform***.

On peut alors définir des ressources versionnées, réutilisées et partagées :

- Composants de bas niveau : CPU/mémoire,stockage, réseau
- Haut-niveau : entrées DNS, fonctionnalités SaaS
- Adopter les déploiements ***GitOps***
- Utilisez GitLab comme stockage d'état Terraform.
- Stockez les modules Terraform

Intégration Terraform

L’intégration s’effectue via Gitlab CI

Un gabarit est fourni qui :

- Utilise ***GitLab-managed Terraform state*** pour stocker les états Terraform
- Déclenche 5 phases de pipeline : ***init, test, validate, build, deploy***
- Exécute les commandes Terraform ***test, validate, plan*** et ***plan-json*** ainsi que ***apply*** dans la branche par défaut
- Effectue un ***IaC scanning*** pour vérifier la sécurité

Exemple *.gitlab-ci.yml*

```
include:
  # To fetch the latest template, use:
  - template: Terraform.latest.gitlab-ci.yml
  # To fetch the advanced latest template, use:
  - template: Terraform/Base.latest.gitlab-ci.yml
  # To fetch the stable template, use:
  - template: Terraform.gitlab-ci.yml
  # To fetch the advanced stable template, use:
  - template: Terraform/Base.gitlab-ci.yml

variables:
  TF_STATE_NAME: default
  TF_CACHE_KEY: default
  # If your terraform files are in a subdirectory, set TF_ROOT accordingly. For
    example:
  # TF_ROOT: terraform/production
```

Kubernetes

GitLab peut se connecter à un cluster Kubernetes pour déployer, gérer et surveiller les applications

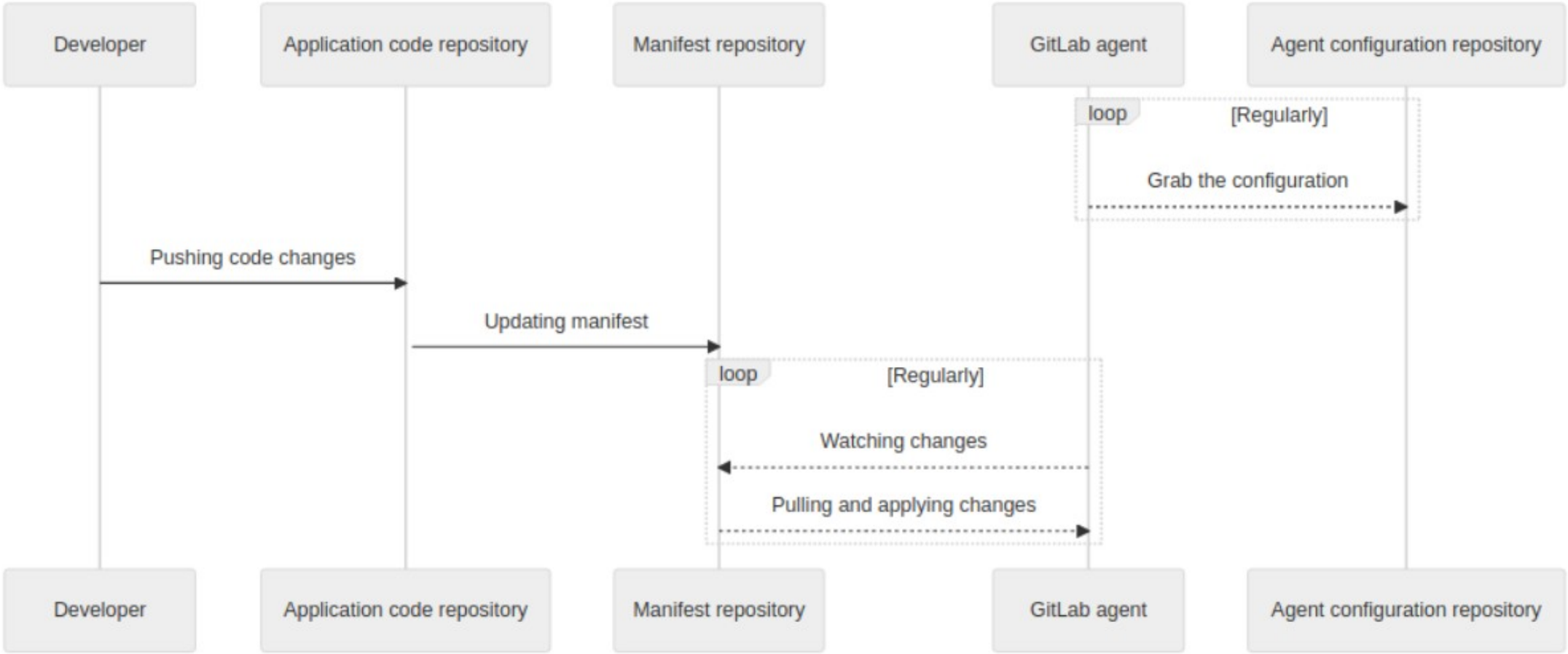
L’interface permet de créer des clusters vers 3 fournisseurs : Google GKE, Amazon EKS, Civo

La connexion s’effectue en installant un agent dans le cluster

2 workflows sont alors proposés :

- GitOps
- Gitlab CI/CD

GitOps


Developer
Application code repository
Manifest repository
GitLab agent
Agent configuration repository
loop
[Regularly]
Grab the configuration
Pushing code changes
Updating manifest
loop
[Regularly]
Watching changes
Pulling and applying changes
Developer
Application code repository
Manifest repository
GitLab agent
Agent configuration repository

Gitlab CI/CD

Dans ce cas, les commandes de déploiement Kubernetes sont dans la pipeline.

La mise en place consiste à

- Enregistrer l’agent dans le projet
- Utiliser des commandes *kubectl* dans *.gitlab-ci.yml*

Exemple

```
deploy:
  image:
    name: bitnami/kubectl:latest
    entrypoint: ['']
  script:
  - kubectl config get-contexts
  - kubectl config use-context path/to/agent/repository:agent-name
  - kubectl apply -f myManifest.yml
```

Support Gitlab pour pipeline standard

Construction et tests développeur
Analyses statiques
Dépôts d’artefacts
Release
Gestion de l’infrastructure
AutoDevOps

Introduction

Auto DevOps fournit une configuration CI / CD prédéfinie qui permet de détecter la nature du projet et d'appliquer un cycle full DevOps automatiquement.

Auto DevOps est activé par défaut sur les projets, il se désactive automatiquement au premier échec de pipeline

Auto DevOps peut également être explicitement activé

Pré-requis

Pré-requis nécessaire :

- ***GitLab Runner*** (Pour toutes les phases) : Doit être configuré pour utiliser Docker ou l'exécuteur Kubernetes and mode privilégié

- ***Base Domain*** (Pour les review apps) : Un domaine configuré avec un DNS * utilisé par tous les projets

- ***Cluster Kubernetes*** avec ***GitLab Agent for Kubernetes*** (agentk) installé

- ***Prometheus*** : Métriques

GitLab Agent for Kubernetes (agentk)

GitLab Agent for Kubernetes est un composant léger déployé dans le cluster Kubernetes.

Il maintient une connexion persistante sortante (gRPC/WebSocket) vers GitLab, plus besoin d'exposer l'API Kubernetes ou de stocker des certificats dans GitLab.

Installation :

```
helm repo add gitlab https://charts.gitlab.io
helm install gitlab-agent gitlab/gitlab-agent \
--namespace gitlab-agent \
--set config.token=<TOKEN> \
--set config.kasAddress=wss://kas.gitlab.com
```

Stratégies

La configuration d'AutoDevOps permet de choisir parmi 3 stratégies de déploiement :

- CD vers la production
- CD incrémentale vers la production (les containers sont progressivement déployés)
- Déploiement automatique vers la pré-prod et déploiement manuel en production

Phases (1)

Auto Build : Crée un build en utilisant un Dockerfile ou les buildpacks Heroku. L'image est poussée est taggé vers le registre de conteneur du projet

Auto Test : Exécute les tests appropriés si il détecte les langages de votre projet

Auto Code Quality : Exécuter une analyse statique et autres vérification du code

Auto SAST : Execute une analyse statique pour détecter des vulnérabilités

Auto Dependency Scanning : Vérifie les dépendances du projet et les éventuelles failles de sécurité

Auto License Management : Génère un rapport sur les dépendances utilisées et leurs licences

Auto Container Scanning : Analyse les failles de sécurité dans les images Docker

Phases (2)

Auto Review Apps : Déploie vers un cluster Kubernetes. Chaque branche obtient un environnement de Review éphémère.

Auto DAST : Test dynamique de la sécurité avec OWASP ZAProxy

Auto Browser Performance Testing : Test de la performance d'une page web avec l'image *Sitespeed.io*

Auto Deploy : Déploiement en production par défaut via Gitlag Agent. Posibilité de faire du canary deployment

Migrations : Possibilité de configuration de scripts de migration Postgres

Personnalisation

Il est possible de personnaliser la pipeline via

- Des buildpacks Heroku personnalisés
- Un Dockerfile personnalisé à la racine du projet
- Des graphiques Helm
- Ou en copiant la configuration dans le fichier de pipeline
  New File → Template AutoDevOps

Template AutoDevOps (1)

```
image: alpine:latest

variables:
  # KUBE_INGRESS_BASE_DOMAIN is the application deployment domain and should be set as a variable at the group or project
     level.
  POSTGRES_USER: user
  POSTGRES_PASSWORD: testing-password
  POSTGRES_ENABLED: "true"
  POSTGRES_DB: $CI_ENVIRONMENT_SLUG
  POSTGRES_VERSION: 9.6.2
  KUBERNETES_VERSION: 1.27.0
  HELM_VERSION: 3.12.0
  DOCKER_DRIVER: overlay2
  ROLLOUT_RESOURCE_TYPE: deployment

stages:
- build
- test
- deploy # dummy stage to follow the template guidelines
- review
- dast
- staging
- canary
- production
- incremental rollout 10%
- incremental rollout 25%
- incremental rollout 50%
- incremental rollout 100%
- performance
- cleanup
```

Template AutoDevOps (2)

```
include:
  - template: Jobs/Build.gitlab-ci.yml
  - template: Jobs/Test.gitlab-ci.yml
  - template: Jobs/Code-Quality.gitlab-ci.yml
  - template: Jobs/Deploy.gitlab-ci.yml
  - template: Jobs/Browser-Performance-Testing.gitlab-ci.yml
  - template: Security/DAST.gitlab-ci.yml
  - template: Security/Container-Scanning.gitlab-ci.yml
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  - template: Security/License-Management.gitlab-ci.yml
  - template: Security/SAST.gitlab-ci.yml

# Override DAST job to exclude main branch
dast:
  except:
    refs:
      - main
```